

دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق

درس یادگیری ماشین

استاد: دکتر امیرحسین نیکوفرد

تمرین سری اول

گروه جاویدنام سپهر ایران

نام و نام خانوادگی	محمدامین محمدیون شبستری
شماره دانشجویی	۴۰۱۲۲۵۰۳
نام و نام خانوادگی	محمدسبحان سخایی
شماره دانشجویی	۴۰۱۱۹۲۵۳
نام و نام خانوادگی	آرین خوشنویس
شماره دانشجویی	۴۰۱۱۷۹۸۳
تاریخ	فروردین ۱۴۰۵



فهرست مطالب

۶	۱ سوال اول
۶	۱.۱ توضیحات شبکه عصبی و معرفی پارامترها
۷	۲.۱ توضیحات Forward Pass
۸	۳.۱ لاس BCE و L2 regularization
۱۱	۴.۱ فرآیند Backpropagation
۱۵	۵.۱ محاسبات بخش (الف)
۱۷	۶.۱ محاسبات بخش (ب)
۲۴	۷.۱ به روز رسانی وزن ها و بایاس ها پس از محاسبه گرادیان ها
۲۵	۲ سوال دوم
۲۶	۱.۲ محو شدگی گرادیان (Vanishing Gradient)
۲۷	۲.۲ پاسخ قسمت الف
۲۸	۳.۲ تفاوت Sigmoid و ReLU در انتشار گرادیان
۳۱	۴.۲ Batch Normalization
۳۲	۵.۲ پاسخ بخش ب
۳۴	۳ سوال سوم
۳۵	۱.۳ توضیح محاسبات Forward به صورت ماتریسی
۳۶	۲.۳ توضیح محاسبات Backward به صورت ماتریسی
۳۸	۳.۳ کد تکمیل شده
۳۹	۴.۳ توضیحات نحوه محاسبه هر قسمت و ساختار Adam
۳۹	۱.۴.۳ ساختار لایه اول و رابطه پیشرو
۴۰	۲.۴.۳ رابطه زنجیره ای برای محاسبه dA_1
۴۰	۳.۴.۳ مشتق تابع ReLU و محاسبه dZ_1
۴۱	۴.۴.۳ محاسبه گرادیان های W_1 و b_1
۴۲	۵.۴.۳ الگوریتم Adam برای به روز رسانی W_1
۴۳	۵.۳ سؤال تحلیلی مرتبط با کد
۴۴	۴ سؤال چهارم
۴۵	۱.۴ توضیح Dropout
۴۵	۲.۴ بخش الف: محاسبه مقدار امید ریاضی خروجی در زمان آموزش
۴۶	۳.۴ بخش ب: رفع ناهمبستگی Train/Test و روش های اصلاح
۴۸	۵ سوال پنجم: کدنویسی برای کمربند حیات
۴۸	۱.۵ فاز اول: اثبات بن بست مدل های خطی



۴۹	فرضیه و مدل ریاضی	۱.۱.۵
۴۹	مرز تصمیم‌گیری (Decision Boundary)	۲.۱.۵
۴۹	تابع هزینه (Cost Function)	۳.۱.۵
۵۰	محدودیت‌ها و عدم تفکیک‌پذیری خطی	۴.۱.۵
۵۰	ارزیابی مدل رگرسیون منطقی و ترسیم مرز تصمیم	۵.۱.۵
۵۱	چالش معماری کمینه (The Minimalist Architect)	۲.۵
۵۱	قضیه تقریب عمومی (Universal Approximation Theorem)	۱.۲.۵
۵۲	ارزیابی مدل MLP روی داده‌های تست	۲.۲.۵
۵۳	مقایسه دو بهینه‌ساز با یکدیگر	۳.۲.۵
۵۳	تحلیل هندسی	۴.۲.۵
۵۴	حالت ۱ نورون	۵.۲.۵
۵۴	حالت ۲ نورون	۶.۲.۵
۵۵	حالت ۳ نورون	۷.۲.۵
۵۵	حالت ۴ نورون	۸.۲.۵
۵۵	نتیجه‌گیری	۹.۲.۵
۵۵	دام توابع فعالسازی (Activation Trap)	۳.۵
۵۵	جزئیات معماری و تنظیمات مدل‌ها	۱.۳.۵
۵۶	روند آموزش و ارزیابی (Training and Evaluation)	۲.۳.۵
۵۷	تفاوت فاحش در سرعت همگرایی	۳.۳.۵
۵۷	پدیده محو شدن گرادیان (Vanishing Gradient Problem)	۴.۳.۵
۵۷	توجیه ریاضی بر اساس مشتقات توابع	۵.۳.۵
۵۷	علت توقف زود هنگام و قطع شدن نمودار Sigmoid	۶.۳.۵
۵۸	نتایج ارزیابی نهایی (دقت روی داده‌های تست)	۷.۳.۵
۵۸	کد تکمیل شده	۴.۵



فهرست تصاویر

۴۸	نمایش دادگان تولید شده در دو بعد	۱
۵۰	ارزیابی مدل رگرسیون منطقی بر روی داده های تست	۲
۵۱	نمایش مرز تصمیم برای مدل رگرسیون منطقی	۳
۵۲	ارزیابی مدل پرسپترون یک لایه با ۴ نورون بر روی مجموعه آموزشی تست	۴
۵۳	ارزیابی مدل با استفاده از بهینه ساز adam بر روی مجموعه تست	۵
۵۴	نمایش مرز تصمیم برای مدل هایی با تعداد نورون های مختلف	۶
۵۶	مقایسه منحنی خطای دو شبکه	۷



فهرست جداول



فهرست برنامه‌ها

۳۸	 کد تابع‌های backward و update adam
۵۸	 نوت بوک تکمیل شده سوال پنج



۱ سوال اول

یک شبکه عصبی چندلایه (MLP) با دو ورودی، یک لایه پنهان شامل دو نورون با تابع فعال‌سازی ReLU و یک لایه خروجی با یک نورون با تابع فعال‌سازی Sigmoid در نظر گرفته شده است. فرض کنید برای یک نمونه آموزشی، بردار ورودی برابر است با:

$$x = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad y = 1$$

و ماتریس‌های وزن و بایاس به صورت زیر مقداردهی اولیه شده‌اند:

$$W^{[1]} = \begin{bmatrix} 0.5 & 1.0 \\ 0.5 & -2.0 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix}$$

$$W^{[2]} = \begin{bmatrix} 1.0 & -1.0 \end{bmatrix}, \quad b^{[2]} = \begin{bmatrix} 0.0 \end{bmatrix}$$

همچنین تابع هزینه از نوع Binary Cross-Entropy (BCE) به همراه منظم‌سازی L2 با ضریب $\lambda = 0.1$ استفاده شده است.

(الف)

مقادیر پیش‌فعال‌سازی لایه پنهان ($z^{[1]}$)، خروجی لایه پنهان ($a^{[1]}$) و پیش‌بینی نهایی ($\hat{y}$) را محاسبه کنید.

(ب)

گرایان کل هزینه (با احتساب ترم منظم‌سازی) نسبت به وزن‌های لایه اول ($W^{[1]}$) و لایه دوم ($W^{[2]}$) را با استفاده از روش پس‌انتشار خطا (Backpropagation) محاسبه کرده و توضیح دهید که چرا مشتق تابع ReLU در بخش‌هایی صفر می‌شود و آیا این موضوع منجر به مشکل Dying ReLU می‌شود یا خیر. همچنین به‌روزرسانی وزن‌ها در الگوریتم گرایان‌کاهی (با استفاده از رابطه شماره ۵۷ اشاره‌شده در درس) را مشخص کنید.

۱.۱ توضیحات شبکه عصبی و معرفی پارامترها

در این مسئله یک شبکه عصبی چندلایه (MLP) با ساختار زیر در نظر گرفته شده است:

- ورودی شامل دو ویژگی:

$$x = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$$

- یک لایه پنهان با دو نورون و تابع فعال‌سازی ReLU.
- یک لایه خروجی با یک نورون و تابع فعال‌سازی Sigmoid.



ماتریس وزن و بایاس لایه اول:

$$W^{[1]} = \begin{bmatrix} 0.5 & 1.0 \\ 0.5 & -2.0 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix}$$

لایه دوم شامل یک نورون:

$$W^{[2]} = \begin{bmatrix} 1.0 & -1.0 \end{bmatrix}, \quad b^{[2]} = \begin{bmatrix} 0.0 \end{bmatrix}$$

توابع فعال‌سازی:

$$\text{ReLU}(z) = \max(0, z)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

تابع هزینه:

$$L = L_{\text{BCE}} + L_{\text{reg}}$$

که در بخش‌های بعدی به طور کامل اثبات می‌شود.

۲.۱ توضیحات Forward Pass

فرآیند Forward Pass مرحله‌ای است که در آن ورودی از لایه ورودی به لایه‌های بعدی منتقل شده، تبدیل می‌شود و نهایتاً خروجی مدل ($\hat{y}$) محاسبه می‌گردد. هدف از این مرحله، محاسبه مقدار پیش‌بینی است تا در ادامه بتوان مقدار هزینه را تعیین کرد.

مرحله اول: محاسبه پیش‌فعال‌سازی لایه پنهان

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

این رابطه از تعریف نورون به دست می‌آید:

$$z = \sum_i w_i x_i + b$$

که در حالت ماتریسی به صورت:

$$z = Wx + b$$

بیان می‌شود.



مرحله دوم: اعمال تابع فعال‌سازی ReLU

$$a^{[1]} = \text{ReLU}(z^{[1]})$$

ReLU باعث می‌شود تنها مقادیر غیر منفی عبور کنند:

$$\text{ReLU}(z) = \begin{cases} z & z > 0 \\ 0 & z \leq 0 \end{cases}$$

مرحله سوم: محاسبه پیش‌فعال‌سازی لایه خروجی

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

این رابطه نیز همانند مرحله قبل از تعریف نوروں به دست می‌آید.

مرحله چهارم: محاسبه پیش‌بینی نهایی با Sigmoid

$$\hat{y} = \sigma(z^{[2]})$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

تابع سیگموئید خروجی را در بازه (0, 1) نگه می‌دارد و برای مسائل دودویی مناسب است.

۳.۱ لاس BCE و L2 regularization

در این مسئله تابع هزینه از دو بخش تشکیل شده است:

$$L = L_{\text{BCE}} + L_{\text{reg}}$$

هدف از این تابع هزینه آن است که:

- میزان خطای مدل در پیش‌بینی مقدار صحیح y اندازه‌گیری شود،
 - و در عین حال مدل از پیچیده شدن بیش از حد و یادگیری نویز داده جلوگیری کند.
- در ادامه هر بخش با دقت کامل توضیح داده می‌شود.

۱. تابع هزینه (BCE) Binary Cross-Entropy

این تابع رایج‌ترین تابع هزینه برای مدل‌هایی است که خروجی آن‌ها احتمال یک کلاس دودویی است. در این مسئله، چون خروجی شبکه:

$$\hat{y} = \sigma(z^{[2]}) \in (0, 1)$$

یک احتمال محسوب می‌شود، بهترین انتخاب تابع BCE است.

تابع BCE به صورت زیر تعریف می‌شود:

$$L_{\text{BCE}} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

چرا این فرم؟ (اثبات دقیق ریاضی) ما فرض می‌کنیم خروجی مدل یک متغیر تصادفی برنولی با پارامتر $\hat{y}$ است. یعنی:

$$P(y = 1|\hat{y}) = \hat{y}, \quad P(y = 0|\hat{y}) = 1 - \hat{y}$$

احتمال مشاهده مقدار واقعی y برابر است با:

$$P(y|\hat{y}) = \hat{y}^y (1 - \hat{y})^{(1-y)}$$

این رابطه نتیجه مستقیم تعریف توزیع برنولی است:

$$y \in \{0, 1\}$$

برای تعریف تابع هزینه از روش (Maximum Likelihood) استفاده می‌کنیم. مدل خوب مدلی است که احتمال مشاهده داده‌های واقعی را بیشینه کند.

پس تابع درست‌نمایی (Likelihood) برابر است با:

$$\mathcal{L} = P(y|\hat{y}) = \hat{y}^y (1 - \hat{y})^{(1-y)}$$

اما در یادگیری ماشین، معمولاً جایگزین بیشینه کردن، کمینه کردن منفی لگاریتم (Likelihood) را استفاده می‌کنیم:

$$L = -\log \mathcal{L}$$

اکنون لگاریتم بگیریم:

$$\log(\mathcal{L}) = y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})$$

پس:

$$L_{\text{BCE}} = -\log(\mathcal{L}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

این دقیقاً همان فرمول BCE است.



چرا BCE مناسب است؟

- خروجی سیگموئید یک احتمال است؛ BCE بر اساس احتمال تعریف شده است.
- اگر مدل اشتباه‌های بدی انجام دهد، هزینه بزرگ می‌شود:

$$\hat{y} \rightarrow 0, y = 1 \Rightarrow L \rightarrow +\infty$$

- مشتق BCE نسبت به $\hat{y}$ بسیار مناسب و پایدار است.
 - این تابع محدب نسبت به $\hat{y}$ است و فرآیند یادگیری را ساده‌تر می‌کند.
- به همین دلیل است که تقریباً تمام شبکه‌های دودویی (از جمله شبکه‌های عمیق، رگرسیون لجستیک، و مدل‌های طبقه‌بندی دودویی) از BCE استفاده می‌کنند.

۲. منظم‌سازی L2 regularization

ترم منظم‌سازی L2 برای جلوگیری از پیچیدگی بیش از اندازه مدل استفاده می‌شود. این ترم هزینه اندازه وزن‌ها را جریمه می‌کند:

$$L_{\text{reg}} = \frac{\lambda}{2} \left(\|W^{[1]}\|_2^2 + \|W^{[2]}\|_2^2 \right)$$

نورم L2 به شکل زیر تعریف می‌شود:

$$\|W\|_2^2 = \sum_{i,j} (W_{ij})^2$$

بنابراین:

$$L_{\text{reg}} = \frac{\lambda}{2} \sum_l \sum_{i,j} (W_{ij}^{[l]})^2$$

چرا از L2 استفاده می‌شود؟ (تحلیل مفهومی)

- وزن‌های بزرگ به این معنی هستند که مدل بیش از حد به برخی ویژگی‌ها وابسته شده است.
- این وابستگی زیاد باعث می‌شود مدل به نویز داده حساس شود (پدیده Overfitting).
- L2 باعث می‌شود وزن‌ها کوچک‌تر بمانند، و مدل «صاف‌تر» و پایدارتر عمل کند.

چرا ضریب $\frac{1}{2}$ داریم؟ برای این که مشتق L2 نسبت به وزن‌ها دقیقاً به شکل ساده زیر باشد:

$$\frac{\partial}{\partial W_{ij}} \left(\frac{1}{2} W_{ij}^2 \right) = W_{ij}$$

این موضوع محاسبات Backpropagation را ساده‌تر می‌کند.



اثبات این که L_2 مدل را پایدار می کند اگر تابع هزینه بدون منظم سازی باشد، مدل سعی می کند:

$$W \rightarrow \infty$$

تا خطا روی داده آموزش صفر شود.

اما با L_2 :

$$L = L_{\text{BCE}} + \frac{\lambda}{2} \sum W^2$$

وقتی وزن ها خیلی بزرگ شوند:

$$\frac{\lambda}{2} \sum W^2 \rightarrow \infty$$

پس مدل مجبور می شود وزن ها را کوچک نگه دارد. این باعث می شود تابع تصمیم صاف تر شود:

$$f(x) = Wx \quad \text{با وزن های کوچک تر} = \text{تغییرات کمتر}$$

در نتیجه:

- حساسیت مدل به نویز کمتر می شود
- نوسان خروجی کاهش پیدا می کند
- مدل بهتر تعمیم می دهد

چرا در این مسئله از $\text{BCE} + L_2$ استفاده شده؟

- BCE بهترین انتخاب برای خروجی احتمالاتی (سیگموئید) و مسائل دودویی است.
- L_2 از رشد بیش از حد وزن ها جلوگیری می کند، خصوصاً در شبکه هایی با تعداد وزن زیاد.
- مقدار $\lambda = 0.1$ نشان می دهد تمرکز مسئله بر جلوگیری از Overfitting بوده است.
- ترکیب $\text{BCE} + L_2$ دقیقاً همان مدل رگرسیون لجستیک منظم سازی شده است که یک مدل استاندارد و بهینه برای classification دوکلاسه است.

۴.۱ فرآیند Backpropagation

الگوریتم Backpropagation روشی برای محاسبه گرادیان تابع هزینه نسبت به تمام وزن ها و بایاس های شبکه است. این الگوریتم قلب آموزش شبکه های عصبی است، زیرا به کمک آن می توان جهت درست حرکت کاهش گرادیان را تعیین کرد. ایده اصلی Backpropagation بر پایه قانون زنجیره ای مشتق گیری (Chain Rule) است. در یک شبکه عصبی، خروجی نهایی به صورت تو در تو به وسیله چندین تابع ساخته شده است؛ بنابراین مشتق گیری مستقیم امکان پذیر نیست، اما با قانون زنجیره ای تمام گرادیان ها مرحله به مرحله محاسبه می شوند.



منطق کلی Backpropagation

در شبکه‌های عصبی:

- هر نورون مقداری را محاسبه می‌کند،
- این مقدار به لایه‌های بعد منتقل می‌شود،
- و نهایتاً خروجی $\hat{y}$ به دست می‌آید.

تابع هزینه L به خروجی نهایی وابسته است و خروجی نهایی نیز به لایه‌های قبلی. پس برای یافتن مشتق $\frac{\partial L}{\partial W^{[l]}}$ باید این وابستگی زنجیره‌ای را باز کنیم.
ساختار محاسبه به صورت زیر است:

$$x \rightarrow z^{[1]} \rightarrow a^{[1]} \rightarrow z^{[2]} \rightarrow a^{[2]} = \hat{y} \rightarrow L$$

هدف Backpropagation حرکت از راست (خروجی) به چپ (لایه‌های قبل) است.

قانون پایه: تعریف دلتا

در Backpropagation ابتدا یک کمیت مهم به نام δ تعریف می‌شود:

$$\delta^{[l]} = \frac{\partial L}{\partial z^{[l]}}$$

این مقدار نشان می‌دهد «تغییر کوچک در پیش‌فعال‌سازی لایه l ، چقدر باعث تغییر در تابع هزینه می‌شود». زمانی که $\delta^{[l]}$ را داشته باشیم، گرادیان وزن‌ها و بایاس‌ها به سادگی به دست می‌آید:

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T$$

$$\frac{\partial L}{\partial b^{[l]}} = \delta^{[l]}$$

پس تمام Backprop به این خلاصه می‌شود که:

$$\delta^{[L]}, \delta^{[L-1]}, \dots, \delta^{[1]}$$

را محاسبه کنیم.

گام اول: محاسبه $\delta^{[L]}$ برای لایه خروجی

برای لایه آخر داریم:

$$a^{[L]} = \sigma(z^{[L]})$$



و تابع هزینه:

$$L_{\text{BCE}} = - \left[y \log(a^{[L]}) + (1 - y) \log(1 - a^{[L]}) \right]$$

مشتق هزینه نسبت به $a^{[L]}$:

$$\frac{\partial L}{\partial a^{[L]}} = - \left(\frac{y}{a^{[L]}} - \frac{1 - y}{1 - a^{[L]}} \right)$$

مشتق سیگموئید:

$$\frac{\partial a^{[L]}}{\partial z^{[L]}} = a^{[L]}(1 - a^{[L]})$$

اما نتیجه‌ی مهم (و بسیار معروف) این است که برای ترکیب Sigmoid + BCE:

$$\delta^{[L]} = a^{[L]} - y$$

این رابطه هم ساده است، هم به لحاظ عددی پایدار.

گام دوم: محاسبه $\delta^{[l]}$ برای لایه‌های قبلی

برای لایه‌های میانی و پنهان از قانون زنجیره‌ای استفاده می‌کنیم:

$$\delta^{[l]} = \frac{\partial L}{\partial z^{[l]}} = \frac{\partial L}{\partial a^{[l]}} \cdot \frac{\partial a^{[l]}}{\partial z^{[l]}}$$

مرحله اول: محاسبه مشتق نسبت به ورودی فعال‌سازی:

$$\frac{\partial L}{\partial a^{[l]}} = (W^{[l+1]})^T \delta^{[l+1]}$$

این رابطه نشان می‌دهد خطا از لایه بعدی به این لایه «برمی‌گردد».

مرحله دوم: ضرب در مشتق تابع فعال‌سازی:

$$\delta^{[l]} = (W^{[l+1]})^T \delta^{[l+1]} \odot f'^{[l]}(z^{[l]})$$

که نماد $\odot$ به معنای ضرب عضو به عضو است.

چرا مشتق ReLU نقش مهمی دارد؟

تابع ReLU:

$$\text{ReLU}(z) = \max(0, z)$$

مشتق آن:



$$f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

پس اگر پیش فعال سازی نوروونی منفی باشد:

$$\delta^{[l]} = 0$$

یعنی نرون در آن نقطه «غیرفعال» است و هیچ گرادیانی عبور نمی دهد. به همین دلیل در شبکه های بزرگ ممکن است برخی نرون ها همیشه در ناحیه $z \leq 0$ باشند و برای همیشه غیرفعال بمانند. این پدیده را Dying ReLU می نامند. در این مسئله، چون فقط یک ورودی داریم، می توان بررسی کرد که کدام نرون فعال یا غیرفعال است.

گام سوم: محاسبه گرادیان وزن ها و بایاس ها

وقتی $\delta^{[l]}$ را داشته باشیم:

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T + \lambda W^{[l]}$$

$$\frac{\partial L}{\partial b^{[l]}} = \delta^{[l]}$$

ترم $\lambda W^{[l]}$ مربوط به منظم سازی L_2 است.

این روابط از تعریف نرون به دست می آیند. برای نرون داریم:

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$$

پس:

$$\frac{\partial z^{[l]}}{\partial W^{[l]}} = a^{[l-1]}$$

و با قانون زنجیره ای:

$$\frac{\partial L}{\partial W^{[l]}} = \frac{\partial L}{\partial z^{[l]}} \frac{\partial z^{[l]}}{\partial W^{[l]}}$$

که دقیقاً نتیجه بالا را می دهد.

گام چهارم: به روز رسانی وزن ها

پس از محاسبه گرادیان ها:

$$W^{[l]} := W^{[l]} - \alpha \frac{\partial L}{\partial W^{[l]}}$$



$$b^{[l]} := b^{[l]} - \alpha \frac{\partial L}{\partial b^{[l]}}$$

که α نرخ یادگیری است.

جمع‌بندی فرآیند Backpropagation

کل فرآیند را می‌توان در چهار گام خلاصه کرد:

۱. محاسبه خروجی با **Forward Pass**.

۲. محاسبه $\delta^{[L]}$ در لایه آخر با استفاده از اختلاف پیش‌بینی و مقدار واقعی.

۳. انتقال خطا به لایه‌های قبل با استفاده از قانون زنجیره‌ای:

$$\delta^{[l]} = (W^{[l+1]})^T \delta^{[l+1]} \odot f^{[l]'}(z^{[l]})$$

۴. محاسبه گرادیان:

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T + \lambda W^{[l]}$$

این الگوریتم به صورت بهینه خطا را از آخر به اول منتقل می‌کند و تمام گرادیان‌ها را با پیچیدگی زمانی خطی نسبت به تعداد وزن‌ها محاسبه می‌کند، که یکی از مهم‌ترین دلایل موفقیت شبکه‌های عمیق است.

۵.۱ محاسبات بخش (الف)

در این بخش هدف آن است که:

- مقادیر پیش‌فعال‌سازی لایه پنهان $z^{[1]}$,
- خروجی لایه پنهان $a^{[1]}$,
- و پیش‌بینی نهایی مدل $\hat{y}$

را به صورت کامل و مرحله به مرحله محاسبه کنیم.

ورودی و پارامترهای داده شده در صورت سؤال به شکل زیر هستند:

$$x = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad W^{[1]} = \begin{bmatrix} 0.5 & 1.0 \\ 0.5 & -2.0 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix}$$

$$W^{[2]} = \begin{bmatrix} 1.0 & -1.0 \end{bmatrix}, \quad b^{[2]} = \begin{bmatrix} 0.0 \end{bmatrix}$$

تابع فعال‌سازی لایه پنهان ReLU و تابع فعال‌سازی لایه خروجی Sigmoid است.



گام اول: محاسبه پیش فعال سازی لایه پنهان $z^{[1]}$ رابطه:

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

ابتدا ضرب ماتریسی $W^{[1]}x$ را محاسبه می کنیم:

$$\begin{aligned} W^{[1]}x &= \begin{bmatrix} 0.5 & 1.0 \\ 0.5 & -2.0 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 0.5(2) + 1.0(-1) \\ 0.5(2) + (-2.0)(-1) \end{bmatrix} \\ &= \begin{bmatrix} 1 - 1 \\ 1 + 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \end{bmatrix} \end{aligned}$$

اکنون بایاس را اضافه می کنیم:

$$z^{[1]} = \begin{bmatrix} 0 \\ 3 \end{bmatrix} + \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ 3.5 \end{bmatrix}$$

گام دوم: اعمال تابع فعال سازی **ReLU** تابع **ReLU** به صورت زیر تعریف می شود:

$$\text{ReLU}(z) = \max(0, z)$$

پس:

$$a^{[1]} = \begin{bmatrix} \max(0, -1.5) \\ \max(0, 3.5) \end{bmatrix} = \begin{bmatrix} 0 \\ 3.5 \end{bmatrix}$$

این خروجی لایه پنهان است.

گام سوم: محاسبه پیش فعال سازی لایه خروجی $z^{[2]}$ رابطه:

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$W^{[2]}a^{[1]} = \begin{bmatrix} 1.0 & -1.0 \end{bmatrix} \begin{bmatrix} 0 \\ 3.5 \end{bmatrix} = 1.0(0) + (-1.0)(3.5) = -3.5$$

بایاس صفر است، پس:

$$z^{[2]} = -3.5$$



گام چهارم: اعمال تابع فعال‌سازی Sigmoid تابع سیگموئید:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

پس:

$$\hat{y} = \sigma(-3.5) = \frac{1}{1 + e^{3.5}}$$

اکنون مقدار عددی:

$$e^{3.5} \approx 33.115$$

$$\hat{y} \approx \frac{1}{1 + 33.115} = \frac{1}{34.115} \approx 0.02930$$

نتیجه نهایی بخش (الف)

$$z^{[1]} = \begin{bmatrix} -1.5 \\ 3.5 \end{bmatrix}, \quad a^{[1]} = \begin{bmatrix} 0 \\ 3.5 \end{bmatrix}, \quad z^{[2]} = -3.5, \quad \hat{y} \approx 0.02930$$

در نتیجه خروجی شبکه (پیش‌بینی مدل) برابر است با:

$$\hat{y} \approx 0.02930$$

۶.۱ محاسبات بخش (ب)

در این بخش هدف آن است که گرادیان‌های وزن‌ها و بایاس‌های دو لایه را با باز کردن کامل قانون زنجیره‌ای (Chain Rule) محاسبه کنیم. یادآوری:

$$x = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad a^{[1]} = \begin{bmatrix} 0 \\ 3.5 \end{bmatrix}, \quad \hat{y} \approx 0.02930, \quad y = 1, \quad \lambda = 0.1$$

گام اول: مشتق تابع هزینه نسبت به خروجی نهایی شبکه

تابع هزینه: BCE

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

مشتق نسبت به $\hat{y}$:

$$\frac{\partial L}{\partial \hat{y}} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right)$$



با توجه به $y = 1$:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{1}{\hat{y}} \approx -\frac{1}{0.02930} \approx -34.13$$

گام دوم: مشتق خروجی لایه خروجی نسبت به پیش فعال سازی لایه خروجی

تابع فعال سازی سیگموئید:

$$\hat{y} = \sigma(z^{[2]}) = \frac{1}{1 + e^{-z^{[2]}}}$$

$$\frac{\partial \hat{y}}{\partial z^{[2]}} = \hat{y}(1 - \hat{y}) \approx 0.02930(1 - 0.02930) \approx 0.02846$$

حال مشتق هزینه نسبت به $z^{[2]}$:

$$\frac{\partial L}{\partial z^{[2]}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{[2]}} = (-34.13)(0.02846) \approx -0.971$$

گام سوم: گرادیان های لایه خروجی

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

مشتق نسبت به وزن های لایه دوم

$$\frac{\partial z^{[2]}}{\partial W_1^{[2]}} = a_1^{[1]} = 0$$

$$\frac{\partial z^{[2]}}{\partial W_2^{[2]}} = a_2^{[1]} = 3.5$$

با قانون زنجیره ای:

$$\frac{\partial L}{\partial W_i^{[2]}} = \frac{\partial L}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial W_i^{[2]}}$$

پس:

$$\frac{\partial L}{\partial W_1^{[2]}} = (-0.971)(0) = 0$$

$$\frac{\partial L}{\partial W_2^{[2]}} = (-0.971)(3.5) \approx -3.399$$

افزودن L_2

$$\frac{\partial L_{\text{total}}}{\partial W_i^{[2]}} = \frac{\partial L_{\text{data}}}{\partial W_i^{[2]}} + \lambda W_i^{[2]}$$

بنابراین:

$$\frac{\partial L_{\text{total}}}{\partial W_1^{[2]}} = 0 + 0.1(1.0) = 0.1$$

$$\frac{\partial L_{\text{total}}}{\partial W_2^{[2]}} = -3.399 + 0.1(-1.0) = -3.499$$

مشتق نسبت به بایاس

$$\frac{\partial z^{[2]}}{\partial b^{[2]}} = 1$$

$$\frac{\partial L}{\partial b^{[2]}} = \frac{\partial L}{\partial z^{[2]}}(1) = -0.971$$

گام چهارم: انتقال مشتق از لایه خروجی به لایه پنهان

مشتق $z^{[2]}$ نسبت به $a^{[1]}$:

$$z^{[2]} = W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]} + b^{[2]}$$

پس:

$$\frac{\partial z^{[2]}}{\partial a_1^{[1]}} = W_1^{[2]} = 1.0$$

$$\frac{\partial z^{[2]}}{\partial a_2^{[1]}} = W_2^{[2]} = -1.0$$

قانون زنجیره‌ای:

$$\frac{\partial L}{\partial a_i^{[1]}} = \frac{\partial L}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a_i^{[1]}}$$

بنابراین:

$$\frac{\partial L}{\partial a_1^{[1]}} = (-0.971)(1.0) = -0.971$$

$$\frac{\partial L}{\partial a_2^{[1]}} = (-0.971)(-1.0) = 0.971$$



گام پنجم: مشتق ReLU

چون:

$$a_1^{[1]} = \max(0, -1.5) = 0$$

$$a_2^{[1]} = \max(0, 3.5) = 3.5$$

مشتق ReLU:

$$\frac{\partial a_1^{[1]}}{\partial z_1^{[1]}} = 0 \quad (\text{نورون اول مُرده است - Dying ReLU})$$

$$\frac{\partial a_2^{[1]}}{\partial z_2^{[1]}} = 1$$

قانون زنجیره‌ای:

$$\frac{\partial L}{\partial z_i^{[1]}} = \frac{\partial L}{\partial a_i^{[1]}} \cdot \frac{\partial a_i^{[1]}}{\partial z_i^{[1]}}$$

پس:

$$\frac{\partial L}{\partial z_1^{[1]}} = (-0.971)(0) = 0$$

$$\frac{\partial L}{\partial z_2^{[1]}} = (0.971)(1) = 0.971$$

پدیده‌ی **Dying ReLU** و اثر آن بر گرادیان‌ها

در لایه پنهان شبکه، تابع فعال‌سازی مورد استفاده ReLU است که به صورت زیر تعریف می‌شود:

$$\text{ReLU}(z) = \max(0, z), \quad \frac{\partial \text{ReLU}(z)}{\partial z} = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}$$

اگر مقدار ورودی یک نورون در لایه پنهان کمتر از صفر باشد، خروجی آن نورون برابر صفر خواهد شد و مهم‌تر از آن،

$$\frac{\partial a}{\partial z} = 0.$$

این حالت باعث رخداد پدیده‌ای به نام **Dying ReLU** می‌شود.تعریف دقیق پدیده **Dying ReLU** وقتی یک نورون ReLU ورودی منفی دریافت کند، خروجی آن صفر می‌شود. اگر این اتفاق به طور

پایدار و برای تعداد زیادی از نمونه‌ها رخ دهد، نورون عملاً «می‌میرد»، زیرا:

- خروجی آن همیشه صفر می‌شود، - گرادیان عبوری از آن همیشه صفر می‌شود، - و این نورون دیگر به فرایند یادگیری کمک نمی‌کند. به عبارت دیگر، مسیر عبور اطلاعات و خطا از این نورون قطع می‌شود.



اثر **Dying ReLU** بر **Backpropagation** از آنجایی که:

$$\frac{\partial a^{[1]}}{\partial z^{[1]}} = 0,$$

طبق قانون زنجیره‌ای:

$$\frac{\partial L}{\partial z^{[1]}} = \frac{\partial L}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} = \frac{\partial L}{\partial a^{[1]}} \cdot 0 = 0.$$

بنابراین:

$$\frac{\partial L}{\partial W_{1j}^{[1]}} = \frac{\partial L}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial W_{1j}^{[1]}} + \lambda W_{1j}^{[1]} = 0 + \lambda W_{1j}^{[1]}.$$

این نتیجه بسیار مهم است:

$$\underbrace{\text{گرادیان داده}}_{\text{از BCE و Backprop}} = 0 \implies \text{تنها نقش باقی مانده در گرادیان} = \lambda W.$$

این محاسبه دقیقاً نشان می‌دهد که نورون اول در لایه پنهان به دلیل مقدار منفی $z_1^{[1]} = -1.5$ کاملاً در حالت **Dying ReLU** قرار گرفته است.

آیا این به معنی توقف کامل به‌روزرسانی وزن‌هاست؟ به‌طور خلاصه:

به‌روزرسانی وزن‌ها کاملاً صفر نمی‌شود.

درست است که «گرادیان داده» (Data Gradient) صفر است، اما به دلیل وجود L2 Regularization:

$$\nabla W = \lambda W$$

و بنابراین:

$$W \leftarrow W - \eta \lambda W.$$

این یعنی:

- وزن‌ها کمی کوچک می‌شوند (Shrink)، - ولی هرگز دقیقاً صفر نمی‌شوند مگر اینکه روند کوچک‌سازی ادامه یابد، - اما هیچ اطلاعاتی از داده‌ها دریافت نمی‌شود و این نورون دیگر یاد نمی‌گیرد.
در نتیجه:

نورون Dying ReLU وزن‌هایش را فقط تحت تأثیر L2 به‌روز می‌کند، نه از داده.

بنابراین:

- یادگیری این نورون متوقف می‌شود،
- مسیر گرادیان از آن نورون قطع می‌شود،
- و این نورون دیگر هیچ تأثیری در تصمیم‌گیری مدل نخواهد داشت.



جمع‌بندی تأثیر Dying ReLU

• اگر ورودی نورون ReLU منفی باشد، مشتق آن صفر است.

• این باعث می‌شود که:

$$\frac{\partial L}{\partial z} = 0$$

• بنابراین یادگیری نورون کاملاً متوقف می‌شود.

• اما L2 باعث یک نوع «کوچک‌سازی وزن» می‌شود:

$$\nabla W = \lambda W$$

• این تنها منبع تغییر وزن باقی مانده است.

• در نبود L2، وزن‌های نورون مرده کاملاً ثابت می‌مانند.

ReLU Dying یعنی توقف کامل گرادیان داده و توقف یادگیری نورون، اما نه لزوماً توقف کامل تغییر وزن (به دلیل L2).

گام ششم: گرادیان‌های لایه اول

$$z_i^{[1]} = W_{i1}^{[1]}x_1 + W_{i2}^{[1]}x_2 + b_i^{[1]}$$

بنابراین:

$$\frac{\partial z_i^{[1]}}{\partial W_{i1}^{[1]}} = x_1 = 2$$

$$\frac{\partial z_i^{[1]}}{\partial W_{i2}^{[1]}} = x_2 = -1$$

قانون زنجیره‌ای:

$$\frac{\partial L}{\partial W_{ij}^{[1]}} = \frac{\partial L}{\partial z_i^{[1]}} \cdot \frac{\partial z_i^{[1]}}{\partial W_{ij}^{[1]}} + \lambda W_{ij}^{[1]}$$

نورون اول:

$$\frac{\partial L}{\partial z_1^{[1]}} = 0$$

پس:

$$\frac{\partial L_{\text{total}}}{\partial W_{11}^{[1]}} = 0 \cdot 2 + 0.1(0.5) = 0.05$$



$$\frac{\partial L_{\text{total}}}{\partial W_{12}^{[1]}} = 0 \cdot (-1) + 0.1(1.0) = 0.1$$

نورون دوم:

$$\frac{\partial L}{\partial z_2^{[1]}} = 0.971$$

$$\frac{\partial L_{\text{total}}}{\partial W_{21}^{[1]}} = 0.971(2) + 0.1(0.5) = 1.942 + 0.05 = 1.992$$

$$\frac{\partial L_{\text{total}}}{\partial W_{22}^{[1]}} = 0.971(-1) + 0.1(-2.0) = -0.971 - 0.2 = -1.171$$

گرادیان بایاس‌های لایه اول

$$\frac{\partial z_i^{[1]}}{\partial b_i^{[1]}} = 1$$

$$\frac{\partial L}{\partial b_1^{[1]}} = 0$$

$$\frac{\partial L}{\partial b_2^{[1]}} = 0.971$$

نتیجه نهایی گرادیان‌ها

$$\nabla W^{[2]} = \begin{bmatrix} 0.1 & -3.499 \end{bmatrix}$$

$$\nabla b^{[2]} = -0.971$$

$$\nabla W^{[1]} = \begin{bmatrix} 0.05 & 0.10 \\ 1.992 & -1.171 \end{bmatrix}$$

$$\nabla b^{[1]} = \begin{bmatrix} 0 \\ 0.971 \end{bmatrix}$$



۷.۱ بهروزرسانی وزن‌ها و بایاس‌ها پس از محاسبه گرادیان‌ها

پس از محاسبه‌ی گرادیان‌های تمام پارامترها، بهروزرسانی وزن‌ها و بایاس‌ها با استفاده از الگوریتم Gradient Descent به صورت زیر انجام می‌شود:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

که در آن:

• θ بیانگر یکی از پارامترهای مدل است (وزن یا بایاس)

• α نرخ یادگیری است

• $\nabla_{\theta} L$ گرادیان محاسبه‌شده هزینه نسبت به آن پارامتر

در ادامه، این بهروزرسانی برای هر لایه به صورت نمادین و سپس با جایگذاری مقادیر عددی نشان داده می‌شود.

۱. بهروزرسانی لایه خروجی

فرمول کلی

$$W^{[2]} \leftarrow W^{[2]} - \alpha \nabla W^{[2]}$$

$$b^{[2]} \leftarrow b^{[2]} - \alpha \nabla b^{[2]}$$

جایگذاری مقادیر عددی گرادیان‌ها گرادیان وزن‌های لایه خروجی:

$$\nabla W^{[2]} = \begin{bmatrix} 0.1 & -3.499 \end{bmatrix}$$

پس:

$$W^{[2]} \leftarrow \begin{bmatrix} 1.0 & -1.0 \end{bmatrix} - \alpha \begin{bmatrix} 0.1 & -3.499 \end{bmatrix}$$

و بایاس:

$$b^{[2]} \leftarrow 0 - \alpha(-0.971)$$

۲. بهروزرسانی لایه پنهان

فرمول کلی

$$W^{[1]} \leftarrow W^{[1]} - \alpha \nabla W^{[1]}$$

$$b^{[1]} \leftarrow b^{[1]} - \alpha \nabla b^{[1]}$$



جایگذاری مقادیر عددی

$$\nabla W^{[1]} = \begin{bmatrix} 0.05 & 0.10 \\ 1.992 & -1.171 \end{bmatrix}$$

$$W^{[1]} \leftarrow \begin{bmatrix} 0.5 & 1.0 \\ 0.5 & -2.0 \end{bmatrix} - \alpha \begin{bmatrix} 0.05 & 0.10 \\ 1.992 & -1.171 \end{bmatrix}$$

برای بایاس‌ها:

$$b^{[1]} \leftarrow \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix} - \alpha \begin{bmatrix} 0 \\ 0.971 \end{bmatrix}$$

۳. تفسیر و نکات مهم

- هر پارامتر به اندازه‌ی α برابر گرادیانش اصلاح می‌شود.
- پارامترهایی که گرادیان آنها بزرگ‌تر است، تغییر بیشتری نسبت به باقی دارند.
- در نوروں دچار Dying ReLU، اثر داده صفر است و تنها بخش L_2 باعث تغییر وزن‌ها می‌شود.
- اگر α بسیار کوچک باشد وزن‌ها به کندی تغییر می‌کنند.
- اگر α بزرگ باشد ممکن است فرآیند آموزش ناپایدار شود.

این بخش، مرحله نهایی محاسبات Backpropagation است و مستقیماً برای اجرای یادگیری استفاده می‌شود.

۲ سوال دوم

صورت سوال:

یک شبکه عمیق MLP با ۵۰ لایه پنهان را در نظر بگیرید که در تمامی لایه‌ها از تابع فعال‌سازی Sigmoid استفاده می‌کند. (الف) با استفاده از قاعده زنجیره‌ای (Chain Rule) و با استناد به مشتق تابع Sigmoid، اثبات کنید که چرا در طول آموزش، ورودی‌های ابتدایی شبکه (نزدیک به ورودی) تقریباً هیچ تغییری نمی‌کنند (فرض کنید ماکزیمم مقدار مشتق Sigmoid برابر با ۰.۲۵ است و مقادیر ورودی‌ها در حدود ۰ هستند).

(ب) فرض کنید برای حل این مشکل، تابع فعال‌سازی را به ReLU تغییر می‌دهیم. یک دانشجوی بی‌دقت، به جای استفاده از Batch Normalization، تمام مقادیر وزن‌ها را با مقادیر تصادفی بسیار بزرگ (مثلاً در بازه [10, 20]) مقداردهی اولیه می‌کند. توضیح دهید که چرا این کار با وجود تابع ReLU منجر به شکست فرایند آموزش می‌شود؟ نقش Batch Normalization (طبق معادله زیر) در جلوگیری از این فاجعه چیست؟

$$z'_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$



۱.۲ محوشدگی گرادیان (Vanishing Gradient)

پدیده محوشدن گرادیان‌ها یا Vanishing Gradient یکی از اساسی‌ترین مشکلات شبکه‌های عصبی عمیق است. این مشکل زمانی رخ می‌دهد که گرادیان خطا در هنگام Backpropagation به لایه‌های ابتدایی شبکه منتقل می‌شود و مقدار آن در هر مرحله ضرب در مشتق تابع فعال‌سازی لایه‌ها می‌گردد. اگر این مشتق‌ها کوچک باشند، گرادیان به مرور کوچک و کوچک‌تر شده و در نهایت تقریباً به صفر می‌رسد. در نتیجه، لایه‌های ابتدایی شبکه تقریباً هیچ‌گاه یاد نمی‌گیرند.

روابط ریاضی محوشدگی گرادیان

فرض کنید یک شبکه عمیق از لایه‌های زیر تشکیل شده باشد و هدف محاسبه گرادیان نسبت به ورودی لایه اول باشد:

$$\frac{\partial L}{\partial z^{(1)}} = \frac{\partial L}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial z^{(L-1)}} \cdot \frac{\partial z^{(L-1)}}{\partial z^{(L-2)}} \cdots \frac{\partial z^{(2)}}{\partial z^{(1)}}$$

اگر تابع فعال‌سازی تمام لایه‌ها Sigmoid باشد، مشتق آن برابر است با:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

مقدار این مشتق همواره در بازه:

$$0 < \sigma'(z) \leq 0.25$$

قرار دارد. بنابراین برای یک شبکه با ۵۰ لایه، گرادیان ورودی لایه اول تقریباً برابر است با:

$$\frac{\partial L}{\partial z^{(1)}} \approx (0.25)^{50}$$

حال مقدار زیر را محاسبه می‌کنیم:

$$(0.25)^{50} \approx 8.88 \times 10^{-31}$$

این عدد تقریباً صفر است؛ بنابراین گرادیان لایه‌های ابتدایی عملاً نابود می‌شود.

چرا این اتفاق می‌افتد؟

- چون Sigmoid برای ورودی‌های بزرگ یا کوچک (Saturation Region) به نواحی اشباع می‌رود.
- در نواحی اشباع، مشتق تقریباً صفر می‌شود.
- گرادیان Backprop در هر لایه ضرب در مشتق تابع فعال‌سازی می‌شود.
- حاصل ضرب تعداد زیادی مقدار کوچک، گرادیان را تقریباً صفر می‌کند.



مشکلات ناشی از محوشدگی گرادیان

- لایه‌های نزدیک ورودی هیچ چیزی یاد نمی‌گیرند.
- وزن‌های این لایه‌ها تقریباً ثابت باقی می‌مانند.
- شبکه تنها لایه‌های انتهایی را یاد می‌گیرد.
- آموزش بسیار کند و ناپایدار می‌شود.
- مدل به دقت مطلوب نمی‌رسد و در سطح پایین «گیر» می‌کند.

۲.۲ پاسخ قسمت الف

در این بخش باید نشان دهیم که شبکه‌ای با ۵۰ لایه Sigmoid دچار محوشدگی شدید گرادیان می‌شود و ورودی‌های ابتدایی تقریباً هیچ تغییری نمی‌کنند.

تحلیل مفهومی

در شبکه‌ای با ۵۰ لایه، گرادیان خروجی باید از ۵۰ مرتبه مشتق تابع Sigmoid عبور کند. چون:

$$\sigma'(z) \leq 0.25$$

پس گرادیان در هر لایه حداقل چهار برابر کوچک‌تر می‌شود. در نهایت مقدار گرادیان:

$$\sim (0.25)^{50}$$

که تقریباً صفر است. در نتیجه، لایه‌های نزدیک به ورودی هیچ یادگیری مؤثری انجام نمی‌دهند.

اثبات ریاضی دقیق

برای لایه k داریم:

$$z^{(k)} = W^{(k)} a^{(k-1)} + b^{(k)}$$

$$a^{(k)} = \sigma(z^{(k)})$$

در Backprop نسبت به $z^{(k)}$ برابر است با:

$$\delta^{(k)} = \frac{\partial L}{\partial z^{(k)}} = \left(W^{(k+1)} \right)^T \delta^{(k+1)} \odot \sigma'(z^{(k)})$$

چون:



$$|\sigma'(z^{(k)})| \leq 0.25$$

بنابراین:

$$\|\delta^{(1)}\| \leq (0.25)^{50} \cdot \|(W^{(2)})^T (W^{(3)})^T \dots (W^{(50)})^T \delta^{(50)}\|$$

جمله نخست یعنی $(0.25)^{50}$ تقریباً صفر است. پس هرچقدر هم که وزن‌ها بزرگ باشند، ضرب در این مقدار کوچک، گرادیان را نابود می‌کند.

نتیجه

ورودی‌های شبکه، یعنی $a^{(1)}$ ، تقریباً هیچ گرادیانی دریافت نمی‌کنند:

$$\nabla_{a^{(1)}} L \approx 0$$

و در نتیجه وزن‌های لایه اول:

$$\Delta W^{(1)} \approx 0$$

یعنی هیچ‌گونه یادگیری در لایه‌های ابتدایی رخ نمی‌دهد. این همان پدیده Vanishing Gradient است که شبکه‌های عمیق مبتنی بر Sigmoid را تقریباً غیرقابل آموزش می‌کند.

۳.۲ تفاوت Sigmoid و ReLU در انتشار گرادیان

تابع‌های فعال‌سازی نقش بسیار مهمی در پایداری گرادیان در شبکه‌های عمیق ایفا می‌کنند. دو تابع مهم و کلاسیک در این زمینه Sigmoid و ReLU هستند. هرکدام رفتار متفاوتی در انتشار گرادیان (Backpropagation) دارند و این تفاوت‌ها سرنوشت یادگیری شبکه‌های عمیق را تعیین می‌کند.

تابع Sigmoid و مشکل محوشدگی گرادیان

تابع Sigmoid به صورت زیر تعریف می‌شود:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

مشتق آن:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$



ویژگی کلیدی: محدود بودن مشتق

$$0 < \sigma'(z) \leq 0.25$$

و این مقدار تنها زمانی به 0.25 نزدیک است که:

$$z \approx 0 \Rightarrow \sigma(z) \approx 0.5$$

اما در نواحی اشباع:

$$z \ll 0 \quad \text{یا} \quad z \gg 0$$

داریم:

$$\sigma'(z) \approx 0$$

اثر روی گرادیان در شبکه‌ای با L لایه:

$$\frac{\partial L}{\partial z^{(1)}} = \left(\prod_{k=2}^L \sigma'(z^{(k)}) W^{(k)} \right) \frac{\partial L}{\partial z^{(L)}}$$

چون هر $\sigma'(z^{(k)}) \leq 0.25$:

$$\prod_{k=2}^L \sigma'(z^{(k)}) \leq (0.25)^L$$

برای L بزرگ، این عبارت تقریباً صفر می‌شود.

نتیجه: مشتق لایه‌های آغازین تقریباً صفر شده و وزن‌های آن لایه‌ها یاد نمی‌گیرند. این همان محو شدن گرادیان است.

تابع ReLU و رفع مشکل محو شدن گرادیان

تابع ReLU:

$$\text{ReLU}(z) = \max(0, z)$$

مشتق آن:

$$\text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

ویژگی کلیدی: داشتن مشتق ۱ در نیمه مثبت در ناحیه فعال:

$$\text{ReLU}'(z) = 1$$

در نتیجه در انتشار گرادیان:

$$\frac{\partial L}{\partial z^{(1)}} \approx \left(\prod_{k=2}^L 1 \cdot W^{(k)} \right) \frac{\partial L}{\partial z^{(L)}}$$

مزیت بزرگ ReLU: هیچ ضریب کوچک‌کننده‌ای مشابه 0.25 سیگمویید وجود ندارد. به همین دلیل شبکه‌های عمیق با ReLU بسیار بهتر آموزش می‌بینند.



مشکلات بزرگ ReLU

تابع ReLU یک تابع فعال‌سازی بسیار پرکاربرد است که به دلیل داشتن مشتق ثابت در نیمه مثبت، جریان گرادیان را در شبکه‌های عمیق پایدارتر می‌کند. با این حال، این تابع دو مشکل اساسی دارد که در صورت عدم کنترل می‌توانند به شکست کامل فرآیند یادگیری منجر شوند: Dying ReLU و Activation Explosion که نهایتاً باعث Gradient Explosion می‌شود.

(۱) مشکل Dying ReLU تعریف تابع:

$$\text{ReLU}(z) = \max(0, z)$$

مشتق آن:

$$\text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

اگر ورودی نورون منفی باشد:

$$z < 0 \Rightarrow a = 0, \quad \text{ReLU}'(z) = 0$$

بنابراین در پس انتشار:

$$\delta = \delta^{(next)} \cdot \text{ReLU}'(z) = 0$$

نتیجه این است که گرادیان آن نورون صفر شده و وزن‌های آن هرگز به‌روزرسانی نمی‌شوند. در این حالت نورون «می‌میرد» و دیگر در طول آموزش فعال نخواهد شد. این پدیده در صورت استفاده از نرخ یادگیری بالا یا وزن‌های اولیه نامناسب تشدید می‌شود.

(۲) مشکل Activation Explosion و سپس Gradient Explosion برخلاف توابعی مانند Sigmoid که خروجی را در بازه‌ای محدود نگه می‌دارند، تابع ReLU در نیمه مثبت فاقد هرگونه محدودیت است:

$$\text{ReLU}(z) = z \quad (z > 0)$$

اگر وزن‌های اولیه بسیار بزرگ باشند، مثلاً:

$$W^{(k)} \in [10, 20]$$

آنگاه مقدار ورودی لایه:

$$z^{(k)} = W^{(k)} a^{(k-1)} + b^{(k)}$$

به‌سرعت بزرگ می‌شود. برای ورودی‌هایی با اندازه واحد:

$$z^{(k)} \approx 10 \text{ تا } 20$$

و در لایه بعد:

$$z^{(k+1)} \approx W^{(k+1)} a^{(k)} \approx (10 \text{ تا } 20) \times (10 \text{ تا } 20) \Rightarrow 100 \text{ تا } 400$$

با عبور از هر لایه مقدار z به‌صورت نمایی رشد می‌کند:

$$z^{(k)} \rightarrow \infty$$

این پدیده «انفجار فعال‌سازی‌ها» نام دارد.



اثر انفجار فعال‌سازی‌ها بر گرادین‌ها در پس‌انتشار، گرادین لایه k برابر است با:

$$\delta^{(k)} = (W^{(k+1)})^T \delta^{(k+1)} \cdot \text{ReLU}'(z^{(k)})$$

چون در نیمه مثبت:

$$\text{ReLU}'(z^{(k)}) = 1$$

پس:

$$\delta^{(k)} \approx W^{(k+1)} \delta^{(k+1)}$$

در شبکه‌های عمیق:

$$|\delta^{(1)}| \approx \left(\prod_{i=1}^L |W^{(i)}| \right) |\delta^{(L)}|$$

اگر وزن‌ها بزرگ باشند، حاصل ضرب بالا نمایی رشد می‌کند:

$$|W|^L \rightarrow \infty$$

در نتیجه:

- گرادین‌ها به شدت بزرگ می‌شوند،
 - وزن‌ها با یک به‌روزرسانی به مقادیر غیرواقعی می‌رسند،
 - مقدار زیان (Loss) به NaN تبدیل می‌شود،
 - آموزش شبکه کاملاً فرو می‌پاشد.
- این مشکل وقتی شدیدتر می‌شود که هیچ سازوکار کنترلی مانند Batch Normalization قبل از ReLU وجود نداشته باشد.

۴.۲ Batch Normalization

روش Batch Normalization یکی از مهم‌ترین تکنیک‌های پایداری آموزش شبکه‌های عمیق است.

ایده اصلی Batch Normalization

مشکل بنیادی شبکه‌های عمیق: **Internal Covariate Shift** یعنی توزیع ورودی لایه‌ها در طول آموزش مدام تغییر می‌کند.

$$z_i^{(k)} = W^{(k)} a^{(k-1)} + b^{(k)}$$

اگر ورودی‌های لایه‌ها تغییرات بزرگی داشته باشند: - گرادین‌ها ناپایدار می‌شوند - شبکه به سختی همگرا می‌شود - ممکن است منفجر یا محو شوند

BatchNorm توزیع ورودی هر لایه را نرمال‌سازی می‌کند.



فرمول Batch Normalization

برای یک مینی بچ $\{z_1, z_2, \dots, z_m\}$:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m z_i$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (z_i - \mu_B)^2$$

نرمال سازی:

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

و سپس مقیاس بندی و انتقال:

$$z'_i = \gamma \hat{z}_i + \beta$$

پارامترهای γ و β قابل یادگیری هستند.

BatchNorm چه مشکلی را حل می کند؟

۱. پایدار کردن توزیع ورودی لایه ها هر لایه ورودی با میانگین صفر و واریانس یک دریافت می کند.
۲. جلوگیری از محوشدگی و انفجار گرادیان مقدار z در محدوده معقول باقی می ماند.
۳. بهبود جریان گرادیان چون ورودی ها نرمال هستند، مشتق ها در مقدار مناسب باقی می مانند.
۴. کار کردن با نرخ یادگیری بزرگ تر چون تغییرات شدید ورودی کنترل شده است.
۵. منظم کنندگی (Regularization) درونی به دلیل نوسانات ناشی از مینی بچ.

—

۵.۲ پاسخ بخش ب

حال به بخش اصلی سؤال می رسیم: چرا مقداردهی وزن ها در بازه $[10^{-4}, 10^{-2}]$ حتی با وجود ReLU باعث شکست کامل شبکه می شود، و BatchNorm چگونه این مشکل را حل می کند؟

مرحله ۱: اثر وزن های بزرگ روی ReLU

ورودی لایه:

$$z^{(k)} = W^{(k)} a^{(k-1)} + b^{(k)}$$

اگر:

$$W^{(k)} \in [10, 20]$$



و ورودی‌ها حدود ۱ باشند:

$$z^{(k)} \approx 10 \text{ تا } 20$$

چون این مقادیر خیلی بزرگ هستند:

$$\text{ReLU}(z^{(k)}) = z^{(k)} \Rightarrow a^{(k)} \text{ بسیار بزرگ}$$

لایه بعد:

$$z^{(k+1)} = W^{(k+1)} a^{(k)} \approx (10 \text{ تا } 20) \times (10 \text{ تا } 20)$$

پس:

$$z^{(k+1)} \approx 100 \text{ تا } 400$$

لایه بعد:

$$z^{(k+2)} \approx 2000 \text{ تا } 8000$$

به سرعت:

$$z \rightarrow \infty$$

این همان انفجار فعال‌سازی‌ها (Activation Explosion) است.

اثر این انفجار روی گرادیان‌ها

در Backprop:

$$\frac{\partial L}{\partial W^{(k)}} = a^{(k-1)} \cdot \delta^{(k)}$$

و:

$$\delta^{(k)} = W^{(k+1)T} \delta^{(k+1)} \cdot \text{ReLU}'(z^{(k)})$$

اگر همه $W^{(k)}$ ها بزرگ باشند:

$$\delta^{(k)} = (10 \text{ تا } 20) \delta^{(k+1)}$$

در نتیجه گرادیان‌ها انفجاری می‌شوند:

$$\prod_{i=1}^L W^{(i)} \rightarrow \infty$$

این یعنی: - وزن‌ها به سرعت می‌ترکند - مقدار Loss به عدد NaN می‌رسد - آموزش کاملاً شکست می‌خورد
بنابراین ReLU مشکل محوشدگی را حل می‌کند اما به شدت مستعد انفجار گرادیان در صورت وزن‌های بزرگ است.



مرحله ۲: نقش Normalization Batch در حل مسئله

BatchNorm قبل از ReLU قرار می‌گیرد:

$$a^{(k)} = \text{ReLU}(\text{BN}(z^{(k)}))$$

چون:

$$\text{BN}(z^{(k)}) = \frac{z^{(k)} - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

حتی اگر:

$$z^{(k)} \approx 1000$$

پس از نرمال‌سازی:

$$\hat{z}^{(k)} \approx \mathcal{N}(0, 1)$$

در نتیجه: - ReLU ورودی‌های کنترل‌شده دارد - فعال‌سازی‌ها در محدوده مناسب باقی می‌مانند - زنجیره گرادیان پایدار می‌ماند - وزن‌ها رشد انفجاری پیدا نمی‌کنند
BatchNorm تضمین می‌کند که هر لایه ورودی مناسب برای یادگیری دریافت کند.

جمع‌بندی پاسخ قسمت ب

- مقداردهی وزن‌ها با اعداد بزرگ باعث تولید z های عظیم می‌شود.
 - این z های بزرگ پس از ReLU باعث ایجاد فعال‌سازی‌های بسیار بزرگ می‌شوند.
 - فعال‌سازی‌های بزرگ باعث گرادیان‌های بزرگ می‌شوند.
 - گرادیان‌های بزرگ باعث انفجار وزن‌ها و شکست آموزش می‌شوند.
 - BatchNorm مقدار z را نرمال می‌کند و از این چرخه جلوگیری می‌کند.
 - بنابراین استفاده از ReLU به تنهایی کافی نیست و راه‌حل درست ترکیب BatchNorm + ReLU است.
- نتیجه نهایی: مقداردهی وزن‌ها در بازه $[10^{-2}, 10^{-1}]$ شبکه را حتی با ReLU نابود می‌کند، اما Normalization Batch این مشکل را با نرمال‌سازی ورودی‌ها رفع می‌کند و مانع انفجار گرادیان می‌شود.

۳ سوال سوم

یکی از مهم‌ترین مهارت‌ها در یادگیری عمیق، توانایی انجام محاسبات Forward Propagation و Backward Propagation برای یک Mini-batch از ورودی‌ها بدون استفاده از حلقه‌های for است. فرض کنید یک Mini-batch با اندازه m داریم و داده‌های ورودی به صورت ماتریس

$$X \in \mathbb{R}^{n_x \times m}$$



آماده شده‌اند، به گونه‌ای که هر ستون $x^{(i)}$ بیانگر یک نمونه از داده است. فرض کنید لایه‌ی پنهان اول دارای n_h نورون باشد، در این صورت خروجی این لایه به صورت:

$$A^{[1]} \in \mathbb{R}^{n_h \times m}$$

خواهد بود. در جریان Backward Propagation نیز مقدار زیر را در اختیار داریم:

$$dZ^{[2]} \in \mathbb{R}^{n_y \times m}$$

که گرادیان تابع هزینه نسبت به ورودی لایه دوم است. در این سؤال، هدف آن است که کلیه‌ی محاسبات Forward و Backward برای این Mini-batch را کاملاً به صورت ماتریسی بیان کنیم و نشان دهیم که چگونه بدون استفاده از حلقه‌ها می‌توان تمام داده‌ها را به صورت هم‌زمان پردازش کرد.

۱.۳ توضیح محاسبات Forward به صورت ماتریسی

در یک لایه‌ی عصبی با وزن‌های

$$W^{[1]} \in \mathbb{R}^{n_h \times n_x}, \quad b^{[1]} \in \mathbb{R}^{n_h \times 1},$$

عملیات Forward روی تمام m داده‌ی ورودی به صورت کاملاً ماتریسی و بدون حلقه انجام می‌شود. ابتدا ورودی‌ها را در قالب ماتریس X داریم:

$$X = \begin{bmatrix} | & | & \dots & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & \dots & | \end{bmatrix} \in \mathbb{R}^{n_x \times m}.$$

مرحله ۱: محاسبه‌ی پیش فعال‌سازی‌ها

برای لایه‌ی اول داریم:

$$Z^{[1]} = W^{[1]}X + b^{[1]}.$$

اثبات سازگاری ابعادی:

$$W^{[1]}X : (n_h \times n_x)(n_x \times m) = n_h \times m.$$

بردار بایاس:

$$b^{[1]} \in \mathbb{R}^{n_h \times 1}$$

از طریق broadcasting روی m ستون تکرار می‌شود:

$$b^{[1]} \rightarrow \begin{bmatrix} b^{[1]} & b^{[1]} & \dots & b^{[1]} \end{bmatrix} \in \mathbb{R}^{n_h \times m}.$$

پس رابطه‌ی نهایی:

$$Z^{[1]} \in \mathbb{R}^{n_h \times m}.$$



مرحله ۲: اعمال تابع فعال‌سازی

تابع فعال‌سازی (مثلاً ReLU یا Sigmoid) به صورت element-wise روی $Z^{[1]}$ اعمال می‌شود:

$$A^{[1]} = g(Z^{[1]}),$$

که در آن:

$$A_{ij}^{[1]} = g(Z_{ij}^{[1]}).$$

ابعاد تغییر نمی‌کند:

$$A^{[1]} \in \mathbb{R}^{n_h \times m}.$$

چرایی اعمال سطر-ستونی (اثبات شهودی): برای هر نورون و هر نمونه یک مقدار مستقل z_{ij} وجود دارد، پس تابع $g(\cdot)$ باید برای هر عنصر جداگانه اعمال شود:

$$g \left(\begin{bmatrix} z_{11} & z_{12} & \cdots \\ z_{21} & z_{22} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix} \right) = \begin{bmatrix} g(z_{11}) & g(z_{12}) & \cdots \\ g(z_{21}) & g(z_{22}) & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$

۲.۳ توضیح محاسبات Backward به صورت ماتریسی

در مرحله‌ی Backward، فرض کنید که گرادیان تابع هزینه نسبت به ورودی لایه دوم یعنی:

$$dZ^{[2]} \in \mathbb{R}^{n_y \times m}$$

را در اختیار داریم. هدف محاسبه‌ی گرادیان‌های زیر است:

$$dW^{[2]}, \quad db^{[2]}, \quad dA^{[1]}, \quad dZ^{[1]}, \quad dW^{[1]}, \quad db^{[1]}.$$

مرحله ۱: محاسبه‌ی گرادیان وزن‌های لایه دوم

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} (A^{[1]})^T.$$

اثبات ماتریسی: هر ستون $dZ^{[2](i)}$ مربوط به نمونه‌ی i است و هر ستون $A^{[1](i)}$ ورودی همین نمونه. گرادیان برای هر وزن از حاصل ضرب مؤلفه‌ای این دو به دست می‌آید. ترکیب همه‌ی نمونه‌ها به صورت ماتریسی:

$$dW^{[2]} = \frac{1}{m} \begin{bmatrix} | & | & & | \\ dZ^{[2](1)} & dZ^{2} & \cdots & dZ^{[2](m)} \\ | & | & & | \end{bmatrix} \begin{bmatrix} --- (A^{1})^T --- \\ --- (A^{[1](2)})^T --- \\ \vdots \\ --- (A^{[1](m)})^T --- \end{bmatrix}.$$

$$\Rightarrow dW^{[2]} \in \mathbb{R}^{n_y \times n_h}.$$



مرحله ۲: گرادیان بایاس لایه دوم

از آنجا که بایاس برای همه‌ی نمونه‌ها یکسان اضافه شده است:

$$db^{[2]} = \frac{1}{m} \sum_{i=1}^m dZ^{[2](i)} = \frac{1}{m} \left[dZ^{[2]} \mathbf{1}_m^T \right],$$

که در آن $\mathbf{1}_m$ یک بردار ستونی از یک‌هاست. ابعاد:

$$db^{[2]} \in \mathbb{R}^{n_y \times 1}.$$

مرحله ۳: محاسبه‌ی گرادیان ورودی لایه اول

$$dA^{[1]} = (W^{[2]})^T dZ^{[2]}.$$

اثبات علت وجود ترانزاده: در Forward داریم:

$$Z^{[2]} = W^{[2]} A^{[1]},$$

پس مشتق زنجیره‌ای:

$$\frac{\partial Z^{[2]}}{\partial A^{[1]}} = (W^{[2]})^T.$$

بنابراین:

$$dA^{[1]} = (W^{[2]})^T dZ^{[2]}.$$

ابعاد:

$$(W^{[2]})^T \in (n_h \times n_y), \quad dZ^{[2]} \in (n_y \times m)$$

پس:

$$dA^{[1]} \in \mathbb{R}^{n_h \times m}.$$

مرحله ۴: محاسبه‌ی dZ لایه اول

اگر تابع فعال‌سازی لایه اول g باشد:

$$dZ^{[1]} = dA^{[1]} \odot g'(Z^{[1]}),$$

که $\odot$ ضرب عضو به عضو است.

مرحله ۵: گرادیان وزن‌های لایه اول

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T.$$



اثبات ابعادی:

$$dZ^{[1]} \in (n_h \times m), \quad X^T \in (m \times n_x)$$

پس:

$$dW^{[1]} \in (n_h \times n_x),$$

که دقیقاً ابعاد وزن لایه اول است.

مرحله ۶: گرادیان بایاس لایه اول

$$db^{[1]} = \frac{1}{m} \sum_{i=1}^m dZ^{[1](i)} = \frac{1}{m} \left[dZ^{[1]} \mathbf{1}_m^T \right].$$

ابعاد:

$$db^{[1]} \in \mathbb{R}^{n_h \times 1}.$$

۳.۳ کد تکمیل شده

```

۱ import numpy as np
۲ def backward_and_adam_update(X, A1, W2, dZ2, W1, b1, m, t, m_t, v_t):
۳     """_summary_
۴
۵     Args:
۶
۷         X (n_x, m): input matrix
۸         A1 (n_h, m): the output of the first hidden layer passed from ReLU
۹         W2 (n_y, n_h): weights of the second layer
۱0        dZ2 (n_y, m): The gradient of the loss function with respect to z2
۱1        W1 (n_h, n_x): weights of the first layer
۱2        b1 (n_h, 1): the bias of the first layer
۱3        m (scaler): the size of mini-batch
۱4        t : current timestamp for adam
۱5        m_t (n_h, n_x): 1st moment for W1
۱6        v_t (n_h, n_x): 2nd moment
۱7
۱8
۱9        # dZ1 - ReLU derivative
۲0        dA1 = W2.T @ dZ2
۲1        dZ1 = dA1 * (A1 > 0)
۲۲

```



```

۲۳     #dW1 , db1
۲۴     dW1 = (1 / m) * (dZ1 @ X.T)
۲۵     db1 = (1 / m) * np.sum(dZ1, axis = 1, keepdims = True)
۲۶
۲۷
۲۸     #adam parameters
۲۹     beta1 = 0.9
۳۰     beta2 = 0.999
۳۱     lr = 0.1
۳۲     epsilon = 1e-8
۳۳
۳۴     #1st moment and 2nd moment updated
۳۵     m_t_new = beta1 * m_t + (1 - beta1) * dW1
۳۶     v_t_new = beta2 * v_t + (1 - beta2) * (dW1 * dW1)
۳۷
۳۸     m_hat = m_t_new / (1 - beta1 ** t)
۳۹     v_hat = v_t_new / (1 - beta2 ** t)
۴۰
۴۱     w1_updated = W1 - lr * (m_hat / (np.sqrt(v_hat) + epsilon))
۴۲
۴۳     return w1_updated, m_t_new, v_t_new
۴۴

```

۴.۳ توضیحات نحوه محاسبه هر قسمت و ساختار Adam

در این زیرقسمت نحوه محاسبه گرادیان‌های لایه اول (Layer 1) و سپس نحوه به‌روزرسانی وزن‌های این لایه با استفاده از الگوریتم Adam را به صورت گام‌به‌گام و با روابط ریاضی کامل توضیح می‌دهیم.

۱.۴.۳ ساختار لایه اول و رابطه پیشرو

فرض کنید ورودی شبکه به صورت ماتریسی با شکل (n_x, m) داده شده است که m تعداد نمونه‌ها و n_x تعداد ویژگی‌ها است. برای لایه اول داریم:

$$Z_1 = W_1 X + b_1, \quad (۱)$$

که در آن:

• $W_1 \in \mathbb{R}^{n_h \times n_x}$ ماتریس وزن‌های لایه اول است،

• $b_1 \in \mathbb{R}^{n_h \times 1}$ بردار بایاس،



• ورودی $X \in \mathbb{R}^{n_x \times m}$ ،

• ورودی قبل از اعمال تابع فعال‌ساز. $Z_1 \in \mathbb{R}^{n_h \times m}$

تابع فعال‌ساز لایه اول را ReLU در نظر می‌گیریم:

$$A_1 = \text{ReLU}(Z_1) = \max(0, Z_1), \quad (2)$$

که $A_1 \in \mathbb{R}^{n_h \times m}$ خروجی لایه اول است.

۲.۴.۳ رابطه زنجیره‌ای برای محاسبه dA_1

در لایه دوم فرض می‌کنیم:

$$Z_2 = W_2 A_1, \quad (3)$$

که $W_2 \in \mathbb{R}^{n_y \times n_h}$ است و $Z_2 \in \mathbb{R}^{n_y \times m}$.

با فرض این‌که گرادیان تابع هزینه نسبت به Z_2 یعنی $dZ_2 = \frac{\partial L}{\partial Z_2}$ را در اختیار داریم، می‌خواهیم گرادیان نسبت به A_1 را به دست آوریم. از قاعده زنجیره‌ای در فضای ماتریسی داریم:

$$dA_1 = \frac{\partial L}{\partial A_1} = \frac{\partial L}{\partial Z_2} \frac{\partial Z_2}{\partial A_1}. \quad (4)$$

از آن‌جا که:

$$Z_2 = W_2 A_1 \Rightarrow \frac{\partial Z_2}{\partial A_1} = W_2, \quad (5)$$

در حالت ماتریسی گرادیان به صورت ضرب W_2^T در dZ_2 ظاهر می‌شود:

$$dA_1 = W_2^T dZ_2. \quad (6)$$

از نظر ابعادی:

$$W_2^T \in \mathbb{R}^{n_h \times n_y}, \quad dZ_2 \in \mathbb{R}^{n_y \times m} \Rightarrow dA_1 \in \mathbb{R}^{n_h \times m}.$$

۳.۴.۳ مشتق تابع ReLU و محاسبه dZ_1

تابع ReLU به صورت زیر تعریف می‌شود:

$$\text{ReLU}(z) = \begin{cases} z, & z > 0, \\ 0, & z \leq 0. \end{cases} \quad (7)$$

بنابراین مشتق آن:

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases} \quad (8)$$



با توجه به این که $A_1 = \text{ReLU}(Z_1)$ است، می توان مشتق را به صورت یک mask تعریف کرد:

$$\text{ReLU}'(Z_1) = \mathbb{I}(Z_1 > 0), \quad (9)$$

که $\mathbb{I}(\cdot)$ تابع نشانگر است و برای هر مولفه عدد ۰ یا ۱ برمی گرداند.

در عمل، چون در مرحله پیشرو A_1 ذخیره شده است، می توانیم از علامت A_1 نیز استفاده کنیم. وقتی $A_1 > 0$ است به این معنی است که $Z_1 > 0$ بوده و مشتق برابر ۱ است، و هنگامی که $A_1 = 0$ است، مشتق صفر خواهد بود. حال با استفاده از قاعده زنجیره ای:

$$dZ_1 = dA_1 \odot \text{ReLU}'(Z_1), \quad (10)$$

که $\odot$ به معنای ضرب عضو به عضو است. بنابراین در پیاده سازی:

$$dZ_1 = dA_1 \odot \mathbb{I}(A_1 > 0). \quad (11)$$

۴.۴.۳ محاسبه گرادیان های W_1 و b_1

رابطه پیشرو لایه اول:

$$Z_1 = W_1 X + b_1. \quad (12)$$

برای هر نمونه ورودی $x^{(i)}$:

$$z_1^{(i)} = W_1 x^{(i)} + b_1. \quad (13)$$

گرادیان نسبت به W_1 برای هر نمونه:

$$\left. \frac{\partial L}{\partial W_1} \right|_{(i)} = dZ_1^{(i)} (x^{(i)})^T. \quad (14)$$

با میانگین گیری روی کل mini-batch با اندازه m داریم:

$$dW_1 = \frac{1}{m} \sum_{i=1}^m dZ_1^{(i)} (x^{(i)})^T. \quad (15)$$

اگر ماتریس $X = [x^{(1)}, x^{(2)}, \dots, x^{(m)}]$ و $dZ_1 = [dZ_1^{(1)}, \dots, dZ_1^{(m)}]$ را در نظر بگیریم، این رابطه به صورت فشرده ماتریسی:

$$dW_1 = \frac{1}{m} dZ_1 X^T. \quad (16)$$

از نظر ابعادی:

$$dZ_1 \in \mathbb{R}^{n_h \times m}, \quad X^T \in \mathbb{R}^{m \times n_x} \Rightarrow dW_1 \in \mathbb{R}^{n_h \times n_x}.$$

هم چنین برای بایاس b_1 ، هر سطر dZ_1 مجموع گرادیان های مربوط به آن نورون روی تمام نمونه ها است، لذا:

$$db_1 = \frac{1}{m} \sum_{i=1}^m dZ_1^{(i)}, \quad (17)$$

که در عمل به صورت جمع روی محور نمونه ها (محور ستون ها) پیاده سازی می شود:

$$db_1 = \frac{1}{m} \sum_{\text{axis}=1} dZ_1. \quad (18)$$

۵.۴.۳ الگوریتم Adam برای بهروزرسانی W_1

الگوریتم Adam ترکیبی از دو ایده اصلی است:

۱. ممنتوم (میانگین نمایی گرادیان‌ها)؛

۲. میانگین نمایی مربع گرادیان‌ها (مشابه RMSProp).

فرض کنید در گام زمانی t ، گرادیان $dW_1^{(t)}$ را در اختیار داریم. ابتدا دو متغیر کمکی (ممنتوم مرتبه اول و دوم) را بهروزرسانی می‌کنیم:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) dW_1^{(t)}, \quad (19)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (dW_1^{(t)})^2, \quad (20)$$

که در آن ضرب‌ها و توان‌ها به صورت عضو به عضو انجام می‌شوند و $\beta_1, \beta_2 \in (0, 1)$ هایپرپارامترهای الگوریتم هستند.

اصلاح بایاس (Bias Correction) به دلیل این‌که در ابتدا m_0 و v_0 معمولاً با صفر مقداردهی اولیه می‌شوند، مقادیر m_t و v_t در گام‌های اولیه تمایل به صفر شدن دارند و این باعث بایاس در تخمین میانگین‌ها می‌شود. برای جبران این اثر، از روابط زیر استفاده می‌کنیم:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad (21)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (22)$$

این روابط را می‌توان به صورت تحلیلی از فرمول میانگین نمایی استخراج کرد و نشان داد که امیدریاضی $\hat{m}_t$ و $\hat{v}_t$ تخمین بدون بایاس میانگین و مربع میانگین گرادیان‌ها هستند.

بهروزرسانی پارامترها در نهایت، بهروزرسانی وزن‌ها بر اساس Adam به صورت زیر انجام می‌شود:

$$W_1^{(t)} = W_1^{(t-1)} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \quad (23)$$

که در آن α نرخ یادگیری (learning rate) و ϵ یک عدد بسیار کوچک (مثلاً 10^{-8}) برای جلوگیری از تقسیم بر صفر است. در پیاده‌سازی این سوال، مقادیر به صورت زیر انتخاب شده‌اند:

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \alpha = 0.01, \quad \epsilon = 10^{-8}.$$

مقادیر استاندارد

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}$$

به صورت گسترده استفاده می‌شوند زیرا تعادلی پایدار میان هموارسازی گرادیان‌ها و کاهش نویز ایجاد می‌کنند. پارامتر β_1 میانگین‌گیری نمایی ممنتوم (میانگین مرتبه اول) را کنترل می‌کند و مقدار حدود 0.9 باعث می‌شود گرادیان‌های تصادفی هموار شوند بدون آن‌که سرعت یادگیری کاهش یابد. پارامتر β_2 مربوط به میانگین‌گیری مرتبه دوم (واریانس) است؛ مقدار بزرگ 0.999 تضمین می‌کند که برآورد واریانس حتی در داده‌های نویزی پایدار بماند. در نهایت، مقدار کوچک ϵ برای جلوگیری از تقسیم بر صفر و تضمین پایداری عددی زمانی که ممنتوم مرتبه دوم بسیار کوچک است، به کار می‌رود.

به طور خلاصه، الگوریتم Adam ابتدا گرادیان dW_1 را دریافت کرده، سپس:



۱. ممنتوم m_t را با میانگین گیری نمایی به روزرسانی می کند،
 ۲. واریانس v_t (میانگین مربع گرادیان) را با میانگین گیری نمایی به روزرسانی می کند،
 ۳. بایاس این دو تخمین را با تقسیم بر $(1 - \beta_1^t)$ و $(1 - \beta_2^t)$ اصلاح می کند،
 ۴. و در نهایت وزن W_1 را با توجه به نسبت $\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ و نرخ یادگیری α به روزرسانی می کند.
- این روش باعث می شود که:

- اطلاعات تاریخی گرادیان ها در تصمیم گیری جهت گام بعدی لحاظ شود (momentum)،
- و اندازه گام برای هر پارامتر به صورت خودکار و متناسب با مقیاس گرادیان آن پارامتر تنظیم شود (adaptive learning rate).

۵.۳ سؤال تحلیلی مرتبط با کد

در محاسبه گرادیان بایاس لایه اول یعنی $db1$ ، چرا هنگام استفاده از دستور `np.sum` نیازمند تنظیم پارامتر `keepdims=True` هستیم؟ اگر این پارامتر تنظیم نشود چه اتفاقی برای Broadcasting در پایتون طی تکرار بعدی می افتد و چرا باعث خطا یا ناهماهنگی ابعادی می شود؟ برای محاسبه بایاس لایه اول در یک شبکه عصبی، معمولاً از رابطه زیر استفاده می شود:

$$db1 = \sum_{i=1}^m dZ1^{(i)},$$

که در آن m تعداد نمونه ها در یک Batch است. در پیاده سازی پایتونی این محاسبه، از دستور `np.sum` استفاده می کنیم:

$$db1 = \text{np.sum}(dZ1, \text{axis} = 1, \text{keepdims}=\text{True}).$$

چرا استفاده از `keepdims` ضروری است؟

زمانی که گرادیان بایاس را محاسبه می کنیم، انتظار داریم شکل (ابعاد) $db1$ به صورت

$$(db1) \in \mathbb{R}^{(n_h, 1)}$$

باقی بماند؛ یعنی یک ستون بردار.

اما اگر بنویسیم:

$$\text{np.sum}(dZ1, \text{axis} = 1)$$

و `keepdims=True` را حذف کنیم، خروجی تبدیل می شود به آرایه ای از شکل:

$$(db1) \rightarrow (n_h,)$$

که یک بردار یک بعدی (بدون بُعد ستونی) است.

در نگاه اول این شاید مشکلی ایجاد نکند، اما در مرحله بعدی یعنی مرحله به روزرسانی:

$$b1 = b1 - \alpha db1,$$



باید $b1$ و $db1$ شکل یکسانی داشته باشند.

از آنجا که:

$$b1 \in \mathbb{R}^{(n_h, 1)},$$

اگر $db1$ برابر با $(n_h,)$ باشد، پایتون باید تلاش کند Broadcasting انجام دهد. اما Broadcasting تنها در شرایطی موفق است که ابعاد از انتها سازگار باشند. یعنی:

$$(n_h, 1) \text{ در برابر } (n_h,)$$

این دو شکل از انتها:

$$(1) \text{ (هیچ) در برابر } (1)$$

سازگار نیستند و پایتون نمی تواند به طور خودکار یک بُعد ستونی اضافه کند.

نتیجه حذف keepdims

اگر $\text{keepdims}=\text{False}$ (پیش فرض) باشد، پیامد فوری آن:

- شکل $db1$ از $(n_h, 1)$ به $(n_h,)$ تغییر می کند.
- Broadcasting هنگام اجرای به روزرسانی پارامترها دچار ناسازگاری می شود.
- در پایتون، معمولاً خطای زیر تولید می شود:

ValueError: operands could not be broadcast together

۴ سؤال چهارم

مطابق صفحه ۵۹ اسلایدها، Dropout در زمان آموزش اعمال شده و در زمان تست (استنتاج) غیرفعال می شود تا خروجی مورد انتظار ثابت بماند. فرض کنید در یک لایه، خروجی یک نورون خاص پیش از اعمال Dropout برابر با $a = 10$ است. نرخ Dropout برابر با $p = 0.5$ (احتمال خاموش شدن نورون) تنظیم شده است.

(الف) مقدار مورد انتظار (Expected Value) خروجی این نورون در طول فاز آموزش چقدر است؟

(ب) در زمان تست (Inference) که Dropout خاموش است، اگر خروجی نورون مستقیماً مقدار $a = 10$ به لایه بعد ارسال شود، ناهماهنگی با فاز آموزش پیدا می کند. برای رفع این مشکل دقیقاً چه عملیات ریاضی باید روی وزن ها یا خروجی این لایه اعمال کنیم؟



۱.۴ توضیح Dropout

Dropout یک روش Regularization برای جلوگیری از Overfitting در شبکه‌های عصبی است. ایده اصلی آن این است که در زمان آموزش، هر نورون با احتمال مشخصی خاموش شود. این کار باعث کاهش Co-Adaptation بین نورون‌ها و بهبود قدرت تعمیم مدل می‌شود.

فرض کنید خروجی یک لایه برابر باشد با

$$A = f(Z)$$

برای اعمال Dropout، ابتدا یک ماسک تصادفی برنولی طبق رابطه زیر تولید می‌کنیم:

$$D \sim \text{Bernoulli}(1 - p)$$

که در آن:

$$D_i = \begin{cases} 1 & \text{با احتمال } (1 - p) \\ 0 & \text{با احتمال } p \end{cases}$$

سپس خروجی Dropout به صورت زیر محاسبه می‌شود:

$$A_{\text{drop}} = A \odot D$$

در نسخه استاندارد مدرن که Dropout Inverted نام دارد، خروجی در زمان آموزش با ضریب $\frac{1}{1-p}$ نیز مقیاس‌بندی می‌شود:

$$A_{\text{train}} = \frac{A \odot D}{1 - p}$$

مزیت این روش آن است که مقدار امید ریاضی خروجی پس از Dropout برابر با مقدار اصلی A باقی می‌ماند:

$$\mathbb{E}[A_{\text{train}}] = A$$

در نتیجه در زمان تست نیاز به هیچ Scaling‌ای نیست:

$$A_{\text{test}} = A$$

اما در نسخه قدیمی‌تر، در زمان آموزش Scaling انجام نمی‌شود و در زمان تست خروجی باید در $(1 - p)$ ضرب شود. این روش امروزه به ندرت استفاده می‌شود.

Dropout علاوه بر Forward، در Backpropagation نیز اعمال می‌شود:

$$dA_{\text{drop}} = dA \odot D$$

نورون‌های خاموش در فاز Forward همچنان گرادیان صفر دریافت می‌کنند.

۲.۴ بخش الف: محاسبه مقدار امید ریاضی خروجی در زمان آموزش

طبق صورت سؤال:

$$a = 10, \quad p = 0.5$$



خروجی نورون پس از Dropout در زمان آموزش یک متغیر تصادفی است:

$$a' = \begin{cases} 0 & \text{با احتمال } p = 0.5 \\ 10 & \text{با احتمال } 1 - p = 0.5 \end{cases}$$

مقدار امید ریاضی خروجی برابر است با:

$$\mathbb{E}[a'] = p \cdot 0 + (1 - p) \cdot a$$

جایگذاری مقادیر:

$$\mathbb{E}[a'] = (0.5) \cdot 0 + (0.5) \cdot 10 = 5$$

$$\boxed{\mathbb{E}[a'] = 5}$$

این یعنی شبکه در فاز آموزش به طور میانگین خروجی ۵ را تجربه کرده است، نه ۱۰. این اختلاف منجر به ایجاد عدم سازگاری میان فاز آموزش و تست می‌شود.

۳.۴ بخش ب: رفع ناهماهنگی Train/Test و روش‌های اصلاح

در زمان تست Dropout خاموش است. بنابراین خروجی نورون همیشه:

$$a_{\text{test}} = 10$$

اما در فاز آموزش مقدار امید ریاضی خروجی:

$$\mathbb{E}[a'] = 5$$

این اختلاف دو برابری سبب ایجاد مشکل **Train/Test Mismatch** می‌شود:

- توزیع ورودی لایه بعد تغییر می‌کند.
- لایه‌ها در فاز آموزش روی مقدار ۵ تنظیم شده‌اند، اما در تست ورودی ۱۰ دریافت می‌کنند.
- مدل دچار Covariate Shift و افزایش ناگهانی فعال‌سازی‌ها می‌شود.

برای رفع این مشکل دو روش وجود دارد:

۱. روش قدیمی **Dropout: Scaling** در زمان تست

چون در آموزش مقدار میانگین برابر ۵ بوده، در تست نیز باید همین مقدار دیده شود، پس:

$$10 \cdot k = 5$$

$$k = \frac{5}{10} = 0.5 = (1 - p)$$



در نتیجه:

$$a_{\text{test-corrected}} = a_{\text{test}} \cdot (1 - p)$$

برای مثال:

$$a_{\text{test-corrected}} = 10 \cdot 0.5 = 5$$

این روش مشکل Train/Test Mismatch را رفع می‌کند اما امروز استاندارد نیست.

۲. روش مدرن: Inverted Dropout (Scaling) در زمان آموزش

در این روش، به جای تغییر خروجی تست، خروجی آموزش را اصلاح می‌کنیم:

$$a' = \begin{cases} 0 & p \\ \frac{a}{1-p} & 1-p \end{cases}$$

برای مسئله حاضر:

$$\frac{10}{1-0.5} = 20$$

پس:

$$a' = \begin{cases} 0 & 0.5 \\ 20 & 0.5 \end{cases}$$

آنگاه:

$$\mathbb{E}[a'] = 0.5 \cdot 0 + 0.5 \cdot 20 = 10$$

نتیجه:

$$\mathbb{E}[a'] = a_{\text{test}}$$

یعنی توزیع آموزش و تست هماهنگ می‌شود و دیگر نیازی به Scaling در زمان تست نیست:

$$a_{\text{test}} = a$$

این روش امروزه در PyTorch، TensorFlow و Keras استفاده می‌شود.

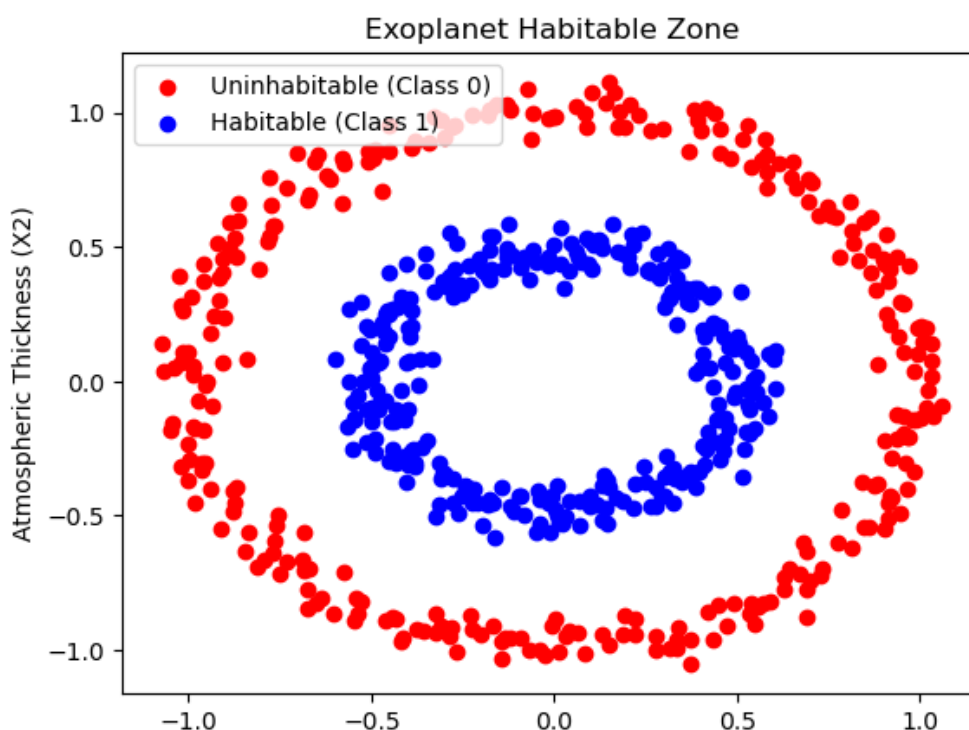
مشکلاتی که با این اصلاح حل می‌شود

- جلوگیری از افزایش ناگهانی خروجی در زمان تست
- هماهنگی میان توزیع‌های آموزش و تست
- جلوگیری از تغییر ورودی لایه‌های بعد (Covariate Shift)
- پایداری یادگیری و عملکرد بهتر مدل
- سازگاری تابع فعال‌سازی‌ها و وزن‌های یادگرفته‌شده



۵ سوال پنجم: کدنویسی برای کمر بند حیات

ابتدا با استفاده از کد داده شده در صورت سوال دیتا را تولید و سپس در دو بعد ترسیم می نماییم. از آنجا که داده های ما دارای دو ویژگی هستند، مشکلی در ترسیم آنها در دو بعد ایجاد نمی شود. شکل زیر داده ها را در دو بعد نمایش می دهد:



شکل ۱: نمایش دادگان تولید شده در دو بعد

همانطور که مشخص است دو کلاس ما دو دایره تو در تو هستند. داده ها با رنگ آبی نمایانگر سیارات با قابلیت حیات می باشند و داده ها با رنگ قرمز نمایانگر سیاراتی هستند که غیر قابل حیات می باشند.

۱.۵ فاز اول: اثبات بن بست مدل های خطی

در این بخش یک مدل LogisticRegression آموزش دادیم.

مدل رگرسیون منطقی (Logistic Regression) علی رغم نامش، یکی از پایه ای ترین و پرکاربردترین الگوریتم های طبقه بندی (Classification) در یادگیری ماشین است. این مدل معمولاً برای مسائل طبقه بندی دودویی (Binary Classification) استفاده می شود، جایی که متغیر هدف (Target) تنها می تواند دو مقدار 0 یا 1 را به خود بگیرد.



۱.۱.۵ فرضیه و مدل ریاضی

در رگرسیون منطقی، هدف ما پیش‌بینی احتمال تعلق یک نمونه به کلاس مثبت (یعنی $y = 1$) است. برای این کار، ابتدا یک ترکیب خطی از ویژگی‌های ورودی ایجاد می‌کنیم:

$$z = \mathbf{w}^T \mathbf{x} + b$$

که در آن $\mathbf{w}$ بردار وزن‌ها، $\mathbf{x}$ بردار ویژگی‌های ورودی و b عرض از مبدأ (Bias) است. از آنجا که خروجی مدل باید یک احتمال بین 0 و 1 باشد، مقدار z از یک تابع فعال‌سازی غیرخطی به نام تابع سیگموئید (Sigmoid) عبور داده می‌شود:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

بنابراین، فرضیه مدل (Hypothesis) به صورت زیر تعریف می‌گردد:

$$\hat{y} = h_{\mathbf{w},b}(\mathbf{x}) = P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

۲.۱.۵ مرز تصمیم‌گیری (Decision Boundary)

مدل زمانی کلاس 1 را پیش‌بینی می‌کند که احتمال محاسبه شده بزرگتر یا مساوی 0.5 باشد. با توجه به ویژگی‌های تابع سیگموئید، این اتفاق زمانی رخ می‌دهد که $z \geq 0$ باشد. بنابراین مرز تصمیم‌گیری مدل به صورت زیر تعریف می‌شود:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

این معادله نشان‌دهنده یک ابرصفحه (Hyperplane) خطی است. به همین دلیل، رگرسیون منطقی یک طبقه‌بند خطی (Linear Classifier) محسوب می‌شود.

۳.۱.۵ تابع هزینه (Cost Function)

برای آموزش مدل، نیازمند تابعی هستیم که میزان خطای پیش‌بینی‌ها را اندازه‌گیری کند. در رگرسیون منطقی از تابع هزینه آنتروپی متقاطع دودویی (Binary Cross-Entropy) یا Loss Log استفاده می‌شود:

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

که در آن m تعداد داده‌های آموزشی، $y^{(i)}$ برچسب واقعی و $\hat{y}^{(i)}$ پیش‌بینی مدل برای نمونه i -ام است. هدف الگوریتم‌های بهینه‌سازی مانند گرادیان کاهشی (Gradient Descent)، یافتن مقادیری برای $\mathbf{w}$ و b است که این تابع هزینه را کمینه کنند.



۴.۱.۵ محدودیت‌ها و عدم تفکیک‌پذیری خطی

بزرگترین محدودیت رگرسیون منطقی این است که تنها قادر به یادگیری مرزهای تصمیم‌گیری خطی است. در صورتی که داده‌ها مانند دیتاست «دایره‌های تو در تو» (Nested Circles) به صورت خطی تفکیک‌پذیر (Non-linearly Separable) نباشند، هیچ خط راستی نمی‌تواند دو کلاس را به درستی از هم جدا کند.

در این شرایط، تابع هزینه نمی‌تواند به مقدار مناسبی همگرا شود و دقت مدل حول و حوش ۵۰ درصد (حدس تصادفی) باقی می‌ماند. برای حل این مشکل هندسی، باید از مدل‌های پیچیده‌تری با قابلیت استخراج ویژگی‌های غیرخطی (مانند شبکه‌های عصبی چندلایه - MLP) استفاده نمود.

۵.۱.۵ ارزیابی مدل رگرسیون منطقی و ترسیم مرز تصمیم

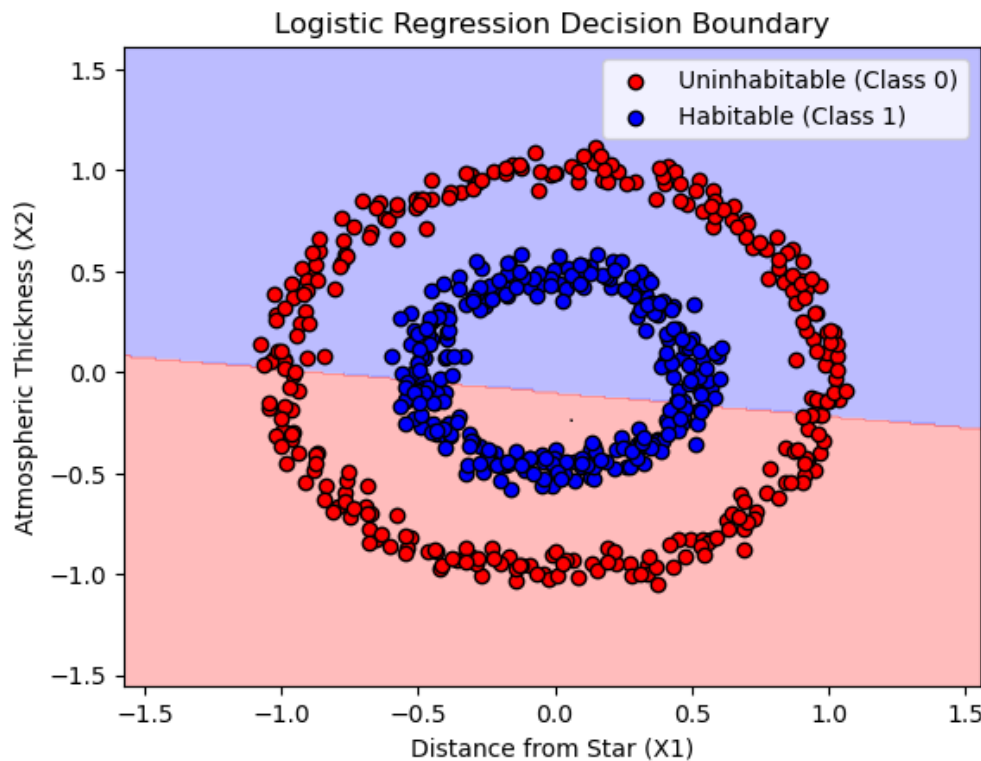
مدل بر روی داده‌های تست مورد ارزیابی قرار گرفت. مقدار دقت مدل بر روی داده‌های تست برابر با 41.67 درصد شد. همچنین گزارش طبقه‌بندی نیز بر روی داده‌های تست گرفته شد. این نتایج به صورت زیر هستند:

accuracy on the test set: 41.67%

	precision	recall	f1-score	support
0	0.42	0.49	0.45	59
1	0.41	0.34	0.37	61
accuracy			0.42	120
macro avg	0.42	0.42	0.41	120
weighted avg	0.42	0.42	0.41	120

شکل ۲: ارزیابی مدل رگرسیون منطقی بر روی داده‌های تست

مرز تصمیم نیز ترسیم شد که شکل آن در زیر آورده شده است:



شکل ۳: نمایش مرز تصمیم برای مدل رگرسیون منطقی

همانطور که مشخص است، مدل رگرسیون منطقی سعی دارد تا این دو کلاس را به صورت خطی از هم تفکیک نماید اما از آنجا که توزیع این داده ها به صورت دایره ای تو در تو می باشد، با شکست مواجه می شود. مرز تصمیم در شکل کاملاً نشان می دهد که مدل به صورت تصادفی عمل کرده و در بهترین حالت ۵۰ درصد داده ها را به درستی طبقه بندی می کند. این موضوع در ارزیابی که بر روی داده های تست انجام دادیم، کاملاً مشخص است.

۲.۵ فاز دوم: چالش معماری کمینه (The Minimalist Architect)

۱.۲.۵ قضیه تقریب عمومی (Universal Approximation Theorem)

قضیه تقریب عمومی (Universal Approximation Theorem) به زبان ساده بیان می کند که یک شبکه عصبی پیش خور (Feedforward Neural Network) که حداقل دارای یک لایه پنهان با تعداد کافی نورون و یک تابع فعال سازی غیر خطی (مانند ReLU یا Sigmoid) باشد، می تواند هر تابع پیوسته ای را با هر دقت دلخواه تقریب بزند.

به بیان ریاضی، اگر رابطه بین ورودی X و خروجی y هر قدر هم پیچیده و غیر خطی باشد، یک شبکه MLP (پرسپترون چندلایه) با تنها یک لایه پنهان و به شرط داشتن تعداد کافی نورون، قادر است آن را بیاموزد و شبیه سازی کند.

اکنون یک شبکه MLP با یک لایه پنهان می سازیم به گونه ای که لایه پنهان دارای تابع فعال ساز Relu و خروجی مدل دارای تابع فعال ساز sigmoid باشد. در کتابخانه sklearn خروجی مدل دارای تابع فعال ساز sigmoid می باشد.

برای جدا کردن یک دایره داخلی از یک دایره خارجی در فضای دوبعدی (X_1, X_2) با استفاده از تابع فعال سازی ReLU، هر نورون



در لایه پنهان مانند یک خط صاف (Hyperplane) در صفحه عمل می‌کند. به طور خاص، خروجی هر نورون از فرم کلی

$$\text{ReLU}(w_1X_1 + w_2X_2 + b)$$

پیروی می‌کند که مرز خطی

$$w_1X_1 + w_2X_2 + b = 0$$

را در فضا ایجاد می‌کند. بنابراین هر نورون متناظر با یک نیم صفحه است و ترکیب چند نورون می‌تواند یک ناحیه بسته را شکل دهد. از نظر هندسی، برای محصور کردن کامل یک ناحیه دوبعدی (مانند ساختن یک چندضلعی که دور دایره داخلی را بگیرد و آن را از دایره بیرونی جدا کند)، حداقل به سه خط مستقل نیاز است؛ یعنی حداقل سه نورون در لایه پنهان. چنین آرایشی می‌تواند یک مثلث حول ناحیه مرکزی بسازد. بنابراین مقدار حداقلی از نظر تئوری برابر با سه نورون است.

با این حال، در عمل مشاهده می‌شود که استفاده از تنها سه نورون نمی‌تواند مرز تصمیم کافی پیچیده و پایدار ایجاد کند. در آزمایش‌ها، مدل با سه نورون تنها به دقت حدود 71.7% بر روی داده‌های تست دست یافت که نشان می‌دهد مرز تصمیم ساخته شده بسیار زمخت بوده و توانایی لازم برای تفکیک کامل دو کلاس را ندارد. این نتیجه از دید کاربردی مطلوب نیست.

برای رفع این مشکل، در کد خود از چهار نورون در لایه پنهان استفاده کرده‌ایم. وجود چهار نورون امکان ساخت یک چندضلعی چهارضلعی (مشابه مربع یا چهارضلعی نامنظم) را فراهم می‌کند که می‌تواند حلقه داخلی را دقیق‌تر تقریب بزند. با استفاده از چهار نورون، شبکه قادر شد ناحیه داخلی را به صورت پایدار محصور کرده و دو کلاس را به خوبی از هم جدا کند. به این ترتیب دقت مدل بر روی داده‌های تست به مقدار بالاتر از 95% رسید که از نظر عملی رضایت‌بخش و قابل اتکا است.

به طور خلاصه، اگرچه سه نورون حداقل مقدار تئوریک برای محاط کردن یک ناحیه بسته در فضای دوبعدی است، اما برای رسیدن به عملکرد مناسب و جداسازی دقیق دو کلاس حلقوی، استفاده از چهار نورون در لایه پنهان عملکرد به مراتب بهتری را ارائه می‌دهد.

۲.۲.۵ ارزیابی مدل MLP روی داده های تست

مدل پرسپترون با یک لایه پنهان را بر روی مجموعه تست، ارزیابی می‌نماییم. نتیجه آن به صورت زیر می‌باشد:

accuracy on the test set: 100.00%

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	59
1	1.00	1.00	1.00	61
accuracy			1.00	120
macro avg	1.00	1.00	1.00	120
weighted avg	1.00	1.00	1.00	120

شکل ۴: ارزیابی مدل پرسپترون یک لایه با ۴ نورون بر روی مجموعه آموزشی تست



همانطور که مشاهده می شود حداقل با ۴ نورون می توانیم به دقت بالای ۹۵ درصد یا در واقع ۱۰۰ درصد دست یابیم که پیشتر به توضیح کامل آن پرداختیم.

۳.۲.۵ مقایسه دو بهینه ساز با یکدیگر

در این مسئله از حل کننده lbfgs استفاده کرده ایم. هنگامی که تعداد نورون های لایه پنهان بسیار کم (برای مثال تنها ۴ نورون) باشد، فضای جست و جوی مدل بسیار محدود و غیر خطی می شود و حل کننده پیش فرض adam معمولاً در مینیمم های محلی گیر می کند و قادر به همگرایی مطلوب نیست. این موضوع باعث می شود که مدل با وجود سادگی مسئله، دقت پایینی به دست آورد یا فرایند آموزش ناپایدار شود.

در مقابل، حل کننده lbfgs که مبتنی بر روش های شبه نیوتنی (Quasi-Newton) است، از اطلاعات تقریبی هسین برای دنبال کردن مسیر بهینه استفاده می کند. این روش به ویژه در دیتاست های کوچک (کمتر از چند هزار نمونه) و معماری های بسیار مینیمال عملکرد فوق العاده ای دارد، زیرا می تواند سطح خطا را با دقت بسیار بالایی جست و جو کرده و از مینیمم های محلی ضعیف عبور کند. در آزمایش های ما، حل کننده adam روی این شبکه کوچک همگرا نشد و در مینیمم های محلی گیر کرد؛ در نتیجه به دقت های پایینی دست یافت. این در حالی است که حل کننده lbfgs به سرعت و به طور پایدار همگرا شد و دقت $\approx 100\%$ روی داده های تست ارائه داد. به طور خلاصه، برای شبکه های بسیار کوچک و دیتاست های کم حجم، استفاده از lbfgs نسبت به adam انتخاب به مراتب مناسب تری است و امکان رسیدن به بیشینه کارایی را فراهم می کند.

در شکل زیر نتایج مربوط به آموزش مدل با استفاده از بهینه ساز adam قابل مشاهده است. در بخش قبل با استفاده از بهینه ساز lbfgs مدل را آموزش دادیم و دقت بالاتری به دست آوردیم.

accuracy on the test set: 93.33%

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.86	0.93	59
1	0.88	1.00	0.94	61
accuracy			0.93	120
macro avg	0.94	0.93	0.93	120
weighted avg	0.94	0.93	0.93	120

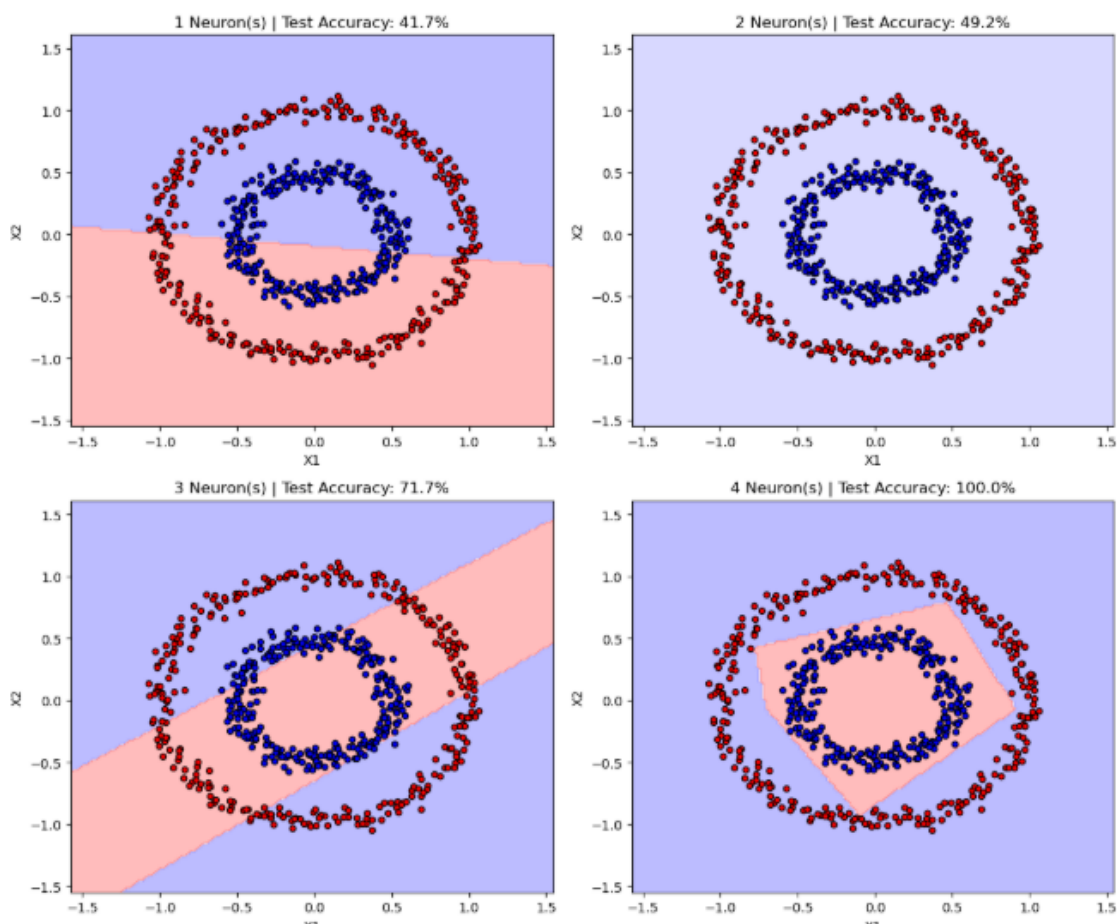
شکل ۵: ارزیابی مدل با استفاده از بهینه ساز adam بر روی مجموعه تست

۴.۲.۵ تحلیل هندسی

در این بخش سوال از ما خواسته که شبکه را با یک، دو، سه و چهار نورون در لایه پنهان آموزش دهیم و مرز تصمیم را برای هر کدام رسم نماییم.



نمودارهای ترسیم شده به همراه دقت آنها بر روی مجموعه تست به صورت زیر به دست آمده است:



شکل ۶: نمایش مرز تصمیم برای مدل هایی با تعداد نرون های مختلف

بر اساس تصویر خروجی مدل، مشاهدات زیر برای هر معماری به دست آمده است:

۵.۲.۵ حالت ۱ نرون

همانطور که در نمودار بالا-چپ مشاهده می شود، یک نرون با تابع فعال سازی ReLU تنها می تواند یک خط صاف (Hyperplane) در فضای دوبعدی ایجاد کند. این خط فضا را به دو نیم صفحه تقسیم می کند. از آنجا که ساختار داده ها به صورت دایره های محیطی و محاطی است، یک خط هرگز قادر به جداسازی آنها نیست و تنها بخشی از داده ها را به صورت تصادفی جدا می کند.

۶.۲.۵ حالت ۲ نرون

از نظر تئوری، دو خط می توانند یک زاویه (شکل ۷) بسازند. اما همانطور که در نمودار بالا-راست مشخص است، مدل نتوانسته حتی یک مرز معنادار برای جداسازی پیدا کند (فضای پس زمینه تقریباً یکپارچه شده است). این امر نشان می دهد که چون ۲ نرون هرگز



نمی‌توانند یک فضای بسته ایجاد کنند، الگوریتم بهینه‌سازی در یافتن وزن‌های مناسب شکست خورده و به یک جواب بسیار ضعیف (کمتر از ۵۰ درصد) همگرا شده است.

۷.۲.۵ حالت ۳ نورون

از منظر ریاضی، ۳ خط متقاطع می‌توانند یک مثلث (یک ناحیه بسته) تشکیل دهند. با این حال، نمودار پایین-چپ نشان می‌دهد که در عمل، شبکه عصبی به جای ساختن یک مثلث کامل، یک نوار یا کانال موازی (Strip) ایجاد کرده است. این نوار دو طرف باز دارد؛ بنابراین توانسته بخشی از دایره داخلی را در بر بگیرد (افزایش دقت به ۷۱٪)، اما از دو انتها دچار نشتی شده و نتوانسته دایره داخلی را کاملاً محصور کند. این نشان می‌دهد که اگرچه ۳ نورون از نظر تئوری حداقل مورد نیاز برای یک شکل بسته است، اما الگوریتم‌های یادگیری ممکن است در عمل به فرم‌های نیمه‌باز زیر بهینه همگرا شوند.

۸.۲.۵ حالت ۴ نورون

نمودار پایین-راست موفقیت کامل مدل را نشان می‌دهد. با در اختیار داشتن ۴ نورون، شبکه توانسته ۴ خط متقاطع را به گونه‌ای تنظیم کند که یک چهارضلعی بسته (Polygon/Diamond) در مرکز فضا شکل بگیرد. این شکل هندسی کاملاً بسته، دایره داخلی را به طور بی‌نقص از دایره خارجی ایزوله کرده و دقت مدل را به ۱۰۰٪ رسانده است. وجود ۴ نورون، درجات آزادی (Degrees of Freedom) کافی را برای الگوریتم بهینه‌سازی فراهم کرده تا بتواند به راحتی یک مرز محدود و محصور پیدا کند.

۹.۲.۵ نتیجه‌گیری

برای دسته‌بندی داده‌هایی که به صورت توپولوژیک در داخل یکدیگر قرار دارند (مانند دایره‌ها)، شبکه عصبی با تابع فعال‌سازی خطی-تکه‌ای (مثل ReLU) موظف است یک چندضلعی بسته بسازد. در حالی که تئوری حداقل ۳ نورون را پیشنهاد می‌دهد، مشاهدات عملی نشان می‌دهد که ۴ نورون پایداری و انعطاف‌پذیری لازم برای تشکیل قطعی یک مرز محصور و رسیدن به دقت کامل را فراهم می‌کند.

۳.۵ فاز سوم: دام توابع فعال‌سازی (Activation Trap)

در این بخش از آزمایش، هدف بررسی تاثیر تابع فعال‌سازی (Activation Function) بر روند آموزش و همگرایی شبکه‌های عصبی با عمق بیشتر است. بدین منظور، دو شبکه عصبی پرسپترون چندلایه (MLP) با معماری کاملاً یکسان تعریف شده‌اند که تنها تفاوت آن‌ها در نوع تابع فعال‌سازی لایه‌های پنهان است.

۱.۳.۵ جزئیات معماری و تنظیمات مدل‌ها

بر اساس کد پیاده‌سازی شده، تنظیمات زیر برای هر دو شبکه در نظر گرفته شده است:

- ساختار لایه‌های پنهان: هر دو مدل دارای ۳ لایه پنهان هستند که در هر لایه ۱۰ نورون قرار دارد ($hidden_layer_sizes = (10, 10, 10)$). این افزایش عمق شبکه به منظور شبیه‌سازی شرایطی است که در آن مشکلات مربوط به گرایان‌ها خود را نشان می‌دهند.



- الگوریتم بهینه‌سازی: از الگوریتم گرادیان کاهشی تصادفی ($solver = 'sgd'$) استفاده شده است تا روند کاهش خطا به صورت پایه و بدون شتاب‌دهنده‌های پیشرفته (مانند Adam) بررسی شود.
- نرخ یادگیری: نرخ یادگیری اولیه روی مقدار 0.01 تنظیم شده است ($learning_rate_init = 0.01$).
- توابع فعال‌سازی: مدل اول ($mlp_sigmoid$) از تابع سیگموئید و مدل دوم (mlp_relu) از تابع ReLU استفاده می‌کند.

۲.۳.۵ روند آموزش و ارزیابی (Training and Evaluation)

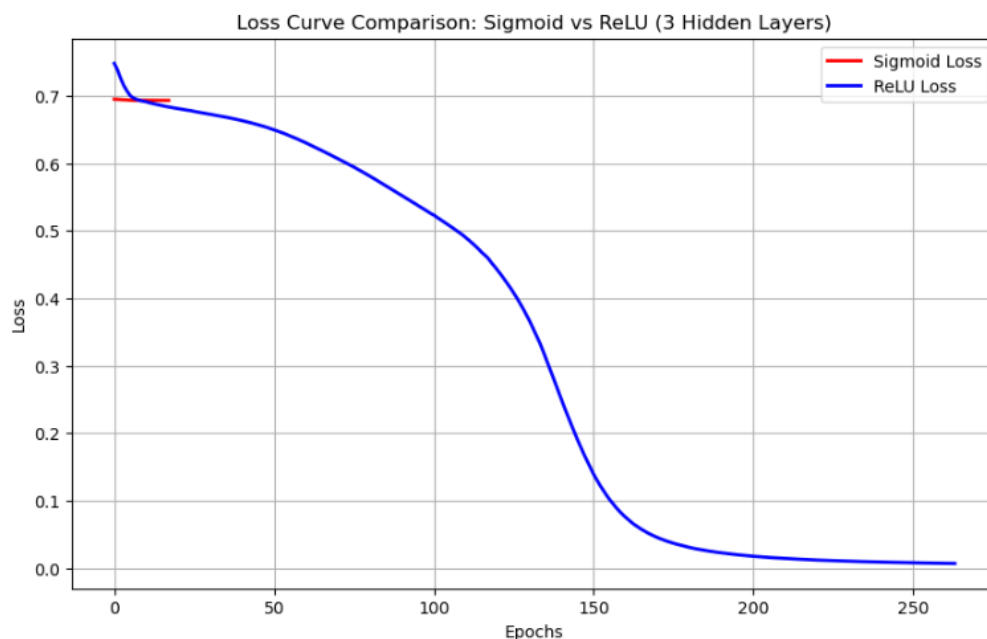
پس از تعریف مدل‌ها، فرآیند آموزش مدل با استفاده از متد $fit()$ بر روی داده‌های آموزشی (X_{train}, y_{train}) انجام می‌شود. کد نوشته شده سه کار اصلی را پس از آموزش انجام می‌دهد:

۱. استخراج منحنی افت خطا (Loss Curve): با استفاده از ویژگی $loss_curve_$ که در کتابخانه scikit-learn تعبیه شده است، مقادیر تابع هزینه در هر تکرار (Epoch) استخراج می‌شود.

۲. رسم نمودار مقایسه‌ای: با استفاده از کتابخانه matplotlib، روند کاهش خطای هر دو مدل در یک نمودار رسم می‌شود. این نمودار به صورت بصری نشان می‌دهد که کدام تابع فعال‌سازی سریع‌تر به حداقل خطا همگرا می‌شود و آیا مدلی در تله‌ی محو شدن گرادیان (Vanishing Gradient) گیر افتاده است یا خیر.

۳. محاسبه دقت نهایی: در نهایت، با استفاده از متد $score()$ ، دقت (Accuracy) هر دو مدل بر روی داده‌های تست (X_{test}, y_{test}) سنجیده و چاپ می‌گردد تا عملکرد نهایی آن‌ها روی داده‌های دیده نشده مقایسه شود.

منحنی افت خطای Loss curve را برای هر دو شبکه در طول اپوک‌ها رسم نمودیم که نتیجه آن در شکل زیر قابل مشاهده است:



شکل ۷: مقایسه منحنی خطای دو شبکه



۳.۳.۵ تفاوت فاحش در سرعت همگرایی

با مشاهده نمودار خروجی، مشخص است که منحنی خطای شبکه ReLU به سرعت (در همان Epoch های اولیه) افت کرده و به سمت صفر میل می‌کند که نشان‌دهنده همگرایی سریع مدل است. در مقابل، خطای شبکه Sigmoid تقریباً به صورت یک خط افقی در مقادیر بالا باقی مانده و در طول آموزش تغییر محسوسی نمی‌کند که نشان‌دهنده عدم همگرایی این مدل می‌باشد.

۴.۳.۵ پدیده محو شدگی گرادیان (Vanishing Gradient Problem)

این مشاهدات مستقیماً به مشکل معروف «محو شدگی گرادیان» در شبکه‌های عصبی اشاره دارد. این مشکل زمانی بارز می‌شود که شبکه‌های عصبی با عمق بیشتر (در این آزمایش ۳ لایه پنهان) را با توابع فعال‌سازی خاصی مانند سیگموید آموزش می‌دهیم.

۵.۳.۵ توجیه ریاضی بر اساس مشتقات توابع

در الگوریتم پس‌انتشار خطا (Backpropagation)، برای به‌روزرسانی وزن‌های شبکه از قاعده زنجیره‌ای (Chain Rule) استفاده می‌شود که مستلزم ضرب مشتقات لایه‌های متوالی در یکدیگر است.

- در شبکه Sigmoid: بیشترین مقدار مشتق تابع سیگموید برابر با 0.25 است. با عبور خطا از ۳ لایه پنهان، این مقادیر کوچک (کمتر از 1) در هم ضرب می‌شوند (به عنوان مثال $0.015 \approx 0.25 \times 0.25 \times 0.25$). در نتیجه، مقدار گرادیان‌ها که به لایه‌های اولیه می‌رسد به شدت به صفر نزدیک شده (محو می‌شود). بدین ترتیب، وزن‌های لایه‌های اولیه به‌روزرسانی نشده و فرآیند یادگیری متوقف می‌گردد.

- در شبکه ReLU: مشتق تابع ReLU برای ورودی‌های مثبت دقیقاً برابر با 1 است. ضرب عدد 1 در خودش طی لایه‌های متعدد، باعث کوچک شدن و تحلیل رفتن گرادیان نمی‌شود. از این رو، گرادیان‌ها بدون افت و محو شدگی تا لایه‌های اولیه پس‌انتشار یافته و شبکه با سرعت بالایی آموزش می‌بیند.

۶.۳.۵ علت توقف زودهنگام و قطع شدن نمودار Sigmoid

قطع شدن نمودار سیگموید پیش از اتمام حداکثر تکرارها، به دلیل رفتار هوشمندانه و مکانیزم‌های درونی کتابخانه scikit-learn (در کلاس MLPClassifier) است که به آن مکانیزم تحمل خطا (Tolerance) یا توقف زودهنگام (Early Stopping) گفته می‌شود. در تنظیمات پیش‌فرض این کتابخانه، پارامتری به نام tol با مقدار پیش‌فرض 0.0001 تعبیه شده است. منطق این الگوریتم به شرح زیر است:

«چنانچه برای تعداد مشخصی از تکرارهای متوالی (مثلاً ۱۰ دوره)، مقدار تابع هزینه (Loss) حداقل به اندازه مقدار tol بهبود نیافت (کاهش پیدا نکرد)، نشان‌دهنده توقف روند یادگیری یا گیر افتادن مدل در یک نقطه است؛ لذا تکرار بیشتر حلقه‌ی آموزش بی‌فایده خواهد بود.»

نتیجه‌گیری نموداری: از آنجا که خطای مدل سیگموید به دلیل مشکل محو شدگی گرادیان کاملاً ثابت مانده بود، الگوریتم پس از گذشت حدود ۲۰ الی ۳۰ Epoch تشخیص داد که شبکه قادر به یادگیری بیشتر نیست و فرآیند آموزش را متوقف کرد. این امر دلیل اصلی قطع شدن زودهنگام خط قرمز رنگ در نمودار است.

بر روی هر دو شبکه، ارزیابی بر روی مجموعه داده تست انجام دادیم که به صورت زیر قابل ارائه می‌باشد:



۷.۳.۵ نتایج ارزیابی نهایی (دقت روی داده‌های تست)

پس از پایان فرآیند آموزش، عملکرد هر دو شبکه بر روی داده‌های آزمون ارزیابی شد. خروجی نهایی مدل‌ها برای سنجش دقت (Accuracy) به شرح زیر است:

• دقت تست در شبکه **Sigmoid**: 49.17%

• دقت تست در شبکه **ReLU**: 100.00%

همان‌طور که از نتایج پیداست، شبکه دارای تابع فعال‌سازی ReLU توانسته است مرز تصمیم‌گیری را به صورت کامل و بی‌نقص یاد بگیرد. در مقابل، شبکه Sigmoid به دلیل توقف زودهنگام ناشی از مشکل محو‌شدگی گرادیان، عملاً در یادگیری الگوهای داده شکست خورده و دقتی معادل با حدس تصادفی (نزدیک به ۵۰ درصد برای دسته‌بندی دودویی) به دست آورده است.

۴.۵ کد تکمیل شده

```

۱  #!/usr/bin/env python
۲  # coding: utf-8
۳
۴  # In[1]:
۵
۶
۷  import numpy as np
۸  import matplotlib.pyplot as plt
۹  from sklearn.datasets import make_circles
۱۰ from sklearn.model_selection import train_test_split
۱۱ from sklearn.linear_model import LogisticRegression
۱۲ from sklearn.metrics import accuracy_score, classification_report
۱۳
۱۴
۱۵ # # Generating Data
۱۶
۱۷ # In[2]:
۱۸
۱۹
۲۰ # Generate the "Habitable Zone" dataset
۲۱ X, y = make_circles(n_samples=600, noise=0.05, factor=0.5, random_state
۲۲ =42)
۲۳ X_train , X_test , y_train , y_test = train_test_split(X, y, test_size=0.2,
۲۴ random_state=42)
۲۵
۲۶

```



```
۲۷ # # Plotting Dataset
۲۸
۲۹ # In[3]:
۳۰
۳۱
۳۲ # Plotting the dataset
۳۳ plt.scatter(X[y==0, 0], X[y==0, 1], color='red', label='Uninhabitable (Class 0)')
۳۴ plt.scatter(X[y==1, 0], X[y==1, 1], color='blue', label='Habitable (Class 1)')
۳۵ plt.xlabel('Distance from Star (X1)')
۳۶ plt.ylabel('Atmospheric Thickness (X2)')
۳۷ plt.legend()
۳۸ plt.title('Exoplanet Habitable Zone')
۳۹ plt.show()
۴۰
۴۱
۴۲ # # Generating logisticRegression Model
۴۳
۴۴ # In[4]:
۴۵
۴۶
۴۷ # Generate model logisticRegression
۴۸ model = LogisticRegression()
۴۹ model.fit(X_train, y_train)
۵۰
۵۱
۵۲ # prediction on model
۵۳ y_pred = model.predict(X_test)
۵۴
۵۵
۵۶ # # Evaluating on model
۵۷
۵۸ # In[11]:
۵۹
۶۰
۶۱ # evaluation on model
۶۲ accuracy = accuracy_score(y_test, y_pred)
۶۳ print(f"accuracy on the test set: {accuracy * 100:.2f}%\n")
۶۴ # classification Report(Precision, Recall, F1-Score)
۶۵ print(classification_report(y_test, y_pred))
۶۶
```



```
۶۷
۶۸ # # Generating meshgrid
۶۹
۷۰ # In[7]:
۷۱
۷۲
۷۳ x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
۷۴ y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
۷۵ xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
۷۶                       np.arange(y_min, y_max, 0.01))
۷۷
۷۸ Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
۷۹ Z = Z.reshape(xx.shape)
۸۰
۸۱
۸۲ # # Plotting decision boundary for logisticRegression
۸۳
۸۴ # In[8]:
۸۵
۸۶
۸۷ plt.contourf(xx, yy, Z, alpha=0.3, cmap='bwr')
۸۸ plt.scatter(X[y==0, 0], X[y==0, 1], color='red', edgecolor='k', label='Uninhabitable (Class 0)')
۸۹ plt.scatter(X[y==1, 0], X[y==1, 1], color='blue', edgecolor='k', label='Habitable (Class 1)')
۹۰
۹۱ plt.xlabel('Distance from Star (X1)')
۹۲ plt.ylabel('Atmospheric Thickness (X2)')
۹۳ plt.title('Logistic Regression Decision Boundary')
۹۴ plt.legend()
۹۵ plt.show()
۹۶
۹۷
۹۸ # In[9]:
۹۹
۱۰۰
۱۰۱ from sklearn.neural_network import MLPClassifier
۱۰۲
۱۰۳
۱۰۴ # # Generating MLP model with one Hidden layer
۱۰۵
۱۰۶ # In[19]:
```



```
۱۰۷
۱۰۸
۱۰۹ # 2.      MLP      "      "      4
۱۱۰ # activation='relu' :
۱۱۱ # solver='lbfgs' :
۱۱۲ mlp_model = MLPClassifier(hidden_layer_sizes=(4,),
۱۱۳                             activation='relu',
۱۱۴                             solver='lbfgs',
۱۱۵                             max_iter=1000,
۱۱۶                             random_state=42)
۱۱۷
۱۱۸
۱۱۹ # # Training Model
۱۲۰
۱۲۱ # In[20]:
۱۲۲
۱۲۳
۱۲۴ mlp_model.fit(X_train, y_train)
۱۲۵
۱۲۶
۱۲۷ # # Prediction on Test set
۱۲۸
۱۲۹ # In[21]:
۱۳۰
۱۳۱
۱۳۲ y_pred = mlp_model.predict(X_test)
۱۳۳
۱۳۴
۱۳۵ # # Evaluation on test set
۱۳۶
۱۳۷ # In[22]:
۱۳۸
۱۳۹
۱۴۰ accuracy = accuracy_score(y_test, y_pred)
۱۴۱ print(f"accuracy on the test set: {accuracy * 100:.2f}%\n")
۱۴۲ print("Classification Report:")
۱۴۳ print(classification_report(y_test, y_pred))
۱۴۴
۱۴۵
۱۴۶ # # Training Mlp model with adam optimization
```



```
۱۴۷
۱۴۸ # In[26]:
۱۴۹
۱۵۰
۱۵۱ mlp_model = MLPClassifier(hidden_layer_sizes=(4,),
۱۵۲                             activation='relu',
۱۵۳                             solver='adam',
۱۵۴                             max_iter=1000,
۱۵۵                             random_state=42)
۱۵۶
۱۵۷
۱۵۸ # # Training Model
۱۵۹
۱۶۰ # In[27]:
۱۶۱
۱۶۲
۱۶۳ mlp_model.fit(X_train, y_train)
۱۶۴
۱۶۵
۱۶۶ # # Prediction on Test set
۱۶۷
۱۶۸ # In[28]:
۱۶۹
۱۷۰
۱۷۱ y_pred = mlp_model.predict(X_test)
۱۷۲
۱۷۳
۱۷۴ # # Evaluation on test set
۱۷۵
۱۷۶ # In[29]:
۱۷۷
۱۷۸
۱۷۹ accuracy = accuracy_score(y_test, y_pred)
۱۸۰ print(f"accuracy on the test set: {accuracy * 100:.2f}%\n")
۱۸۱ print("Classification Report:")
۱۸۲ print(classification_report(y_test, y_pred))
۱۸۳
۱۸۴
۱۸۵ # # Plotting Decision Boundary for neurons
۱۸۶
```



```
۱۸۷ # In[24]:
۱۸۸
۱۸۹
۱۹۰ fig, axes = plt.subplots(2, 2, figsize=(12, 10))
۱۹۱ neurons = [1, 2, 3, 4]
۱۹۲
۱۹۳ for i, ax in enumerate(axes.ravel()):
۱۹۴     n = neurons[i]
۱۹۵
۱۹۶     # n
۱۹۷     mlp = MLPClassifier(hidden_layer_sizes=(n,),
۱۹۸                         activation='relu',
۱۹۹                         solver='lbfgs',
۲۰۰                         max_iter=2000,
۲۰۱                         random_state=42)
۲۰۲     mlp.fit(X_train, y_train)
۲۰۳
۲۰۴     # (Meshgrid)
۲۰۵     Z = mlp.predict(np.c_[xx.ravel(), yy.ravel()])
۲۰۶     Z = Z.reshape(xx.shape)
۲۰۷
۲۰۸     #
۲۰۹     acc = mlp.score(X_test, y_test)
۲۱۰
۲۱۱     #
۲۱۲     ax.contourf(xx, yy, Z, alpha=0.3, cmap='bwr')
۲۱۳     ax.scatter(X[y==0, 0], X[y==0, 1], color='red', edgecolor='k', s=20)
۲۱۴     ax.scatter(X[y==1, 0], X[y==1, 1], color='blue', edgecolor='k', s=20)
۲۱۵
۲۱۶     ax.set_title(f'{n} Neuron(s) | Test Accuracy: {acc*100:.1f}%')
۲۱۷     ax.set_xlabel('X1')
۲۱۸     ax.set_ylabel('X2')
۲۱۹
۲۲۰ plt.tight_layout()
۲۲۱ plt.show()
۲۲۲
۲۲۳
۲۲۴ # # Training models with sigmoid or Relu
۲۲۵
۲۲۶ # In[25]:
```




```
۲۲۷
۲۲۸
۲۲۹ mlp_sigmoid = MLPClassifier(hidden_layer_sizes=(10, 10, 10),
۲۳۰                             activation='logistic',
۲۳۱                             solver='sgd',
۲۳۲                             learning_rate_init=0.01,
۲۳۳                             max_iter=1000,
۲۳۴                             random_state=42)
۲۳۵
۲۳۶
۲۳۷ mlp_relu = MLPClassifier(hidden_layer_sizes=(10, 10, 10),
۲۳۸                             activation='relu',
۲۳۹                             solver='sgd',
۲۴۰                             learning_rate_init=0.01,
۲۴۱                             max_iter=1000,
۲۴۲                             random_state=42)
۲۴۳
۲۴۴
۲۴۵ print( "           Sigmoid...")
۲۴۶ mlp_sigmoid.fit(X_train, y_train)
۲۴۷
۲۴۸ print( "           ReLU...")
۲۴۹ mlp_relu.fit(X_train, y_train)
۲۵۰
۲۵۱
۲۵۲ plt.figure(figsize=(10, 6))
۲۵۳ plt.plot(mlp_sigmoid.loss_curve_, label='Sigmoid Loss', color='red', linewidth=2)
۲۵۴ plt.plot(mlp_relu.loss_curve_, label='ReLU Loss', color='blue', linewidth=2)
۲۵۵
۲۵۶ plt.title('Loss Curve Comparison: Sigmoid vs ReLU (3 Hidden Layers)')
۲۵۷ plt.xlabel('Epochs')
۲۵۸ plt.ylabel('Loss')
۲۵۹ plt.legend()
۲۶۰ plt.grid(True)
۲۶۱ plt.show()
۲۶۲
۲۶۳
۲۶۴ print(f"Test Accuracy (Sigmoid): {mlp_sigmoid.score(X_test, y_test)*100:.2f}%")
۲۶۵ print(f"Test Accuracy (ReLU): {mlp_relu.score(X_test, y_test)*100:.2f}%")
```



--